PersonalDotFiles - NixOS Edition

mrtaichi

October 24, 2026

Contents

1 Introducción **4**

2 Especificación **4**

3 NixOS **4**
3.1 Instalación y Sistema de Archivos 5
3.2 Configuraciones globales 7
3.3 Cuentas de usuario 8
3.4 Utilidades principales 8
3.4.1 System Packages 9
3.4.2 User Packages 10
3.5 Flakes 11
3.6 Cron 12
3.7 SSH 13
3.8 Configuraciones privadas 14
3.9 stateVersion 14

4 OhMyZSH **15**
4.1 Instalación 15
4.2 Configuraciones preliminares 16
4.3 Aliases 17
4.3.1 Sudo 17
4.3.2 Arduino CLI 17
4.4 PATH 18
4.5 Terminal 19

5 Doom Emacs **19**
5.1 Introducción . . . 19
5.2 Instalación . . . 19
5.3 init.el . . . 20
5.3.1 Módulos Input . . . 26
5.3.2 Módulos Completion . . . 26
5.3.3 Módulos UI . . . 26
5.3.4 Módulos Editor . . . 27
5.3.5 Módulos Emacs . . . 27
5.3.6 Módulos Term . . . 28
5.3.7 Módulos Checkers . . . 28
5.3.8 Módulos Tools . . . 28
5.3.9 Módulos OS . . . 29
5.3.10 Módulos de lenguajes . . . 29
5.3.11 Módulos de Email . . . 30
5.3.12 Módulos de Apps . . . 31
5.3.13 Módulos de configuración . . . 31
5.4 config.el . . . 32
5.4.1 Estética y estilo . . . 32
5.4.2 Org Mode . . . 33
5.4.3 Atomic Chrome . . . 36
5.4.4 LaTeX . . . 37
5.4.5 Programación Letrada . . . 37
5.5 packages.el . . . 38
5.6 Pegar imágenes del portapapeles . . . 38

6 Hyprland **41**
6.1 Instalación . . . 41
6.2 Hy3 . . . 43
6.3 Configuraciones (nuevo) . . . 43
6.4 Configuraciones . . . 52
6.4.1 Configuración de los monitores . . . 52
6.4.2 Defaults . . . 53
6.4.3 Wallpaper . . . 53
6.4.4 Waybar . . . 54
6.4.5 Variables de entorno . . . 68
6.4.6 . . . 69

7 Dropbox **69**
7.1 Instalación . . . 69

8 Virtual Box y QEMU/KVM **69**

9 Zen Browser **70**
9.1 Instalación 70

10 Programación con IA **70**
10.1 Gptel 70
10.2 OpenCode 72
10.3 Copilot.el 72

11 Lenguajes de programación **73**
11.1 C y C++ 73
11.2 Golang 73
11.3 Javascript 74
11.4 Python 74
11.4.1 Pyenv 75
11.4.2 Conda 75
11.5 Haskell 76
11.6 Clojure 76
11.7 Java 77

12 Ciberseguridad **77**
12.1 Suite de herramientas 78
12.1.1 Proxies 78
12.1.2 Port Scanners 79
12.1.3 Web 79
12.1.4 Generic 81
12.1.5 Services 82
12.1.6 Password 84
12.1.7 Exploits 84
12.1.8 Fuzzers 84
12.2 Maquina virtual 85
12.2.1 Flujo de trabajo con el contenedor de Kali Linux . . . 86
12.2.2 QoL (Calidad de vida) 86
12.3 Flujo de trabajo de Ciberseguridad 87

13 VPNs **87**

14 Conclusión **88**

1 Introducción

Bienvenido a mis PersonalDotFiles, la recopilación de configuraciones y dotfiles de mi sistema operativo NixOS. En este documento, exploraré las diferentes herramientas y configuraciones que utilicé para crear un entorno de trabajo personalizado y eficiente.

Comenzaré explicando mi flujo de trabajo y las razones detrás de mi elección de NixOS como sistema operativo. Luego, profundizaré en las configuraciones específicas de mi sistema, incluyendo la instalación y configuración de NixOS, la creación de un sistema de archivos BTRFS, y la configuración de Systemd-boot como gestor de arranque.

Además, exploraré las herramientas y configuraciones que utilicé para crear un entorno de trabajo productivo, incluyendo la configuración de OhMyZSH, Doom Emacs, Hyprland, y otras herramientas esenciales para mi flujo de trabajo. También cubriré la configuración de los paquetes y dependencias necesarias para cada herramienta, utilizando NixOS y su gestor de paquetes.

2 Especificación

El flujo de trabajo y configuración que explica este documento es el que utilizo para las encarnaciones de mis máquinas de trabajo principales.

Como estudiante de Ciencias de la Computación que además tiene bastantes intereses que lo llevan a trabajar en un par de cosillas a la vez, no me siento seguro de afirmar que mi marco de trabajo es el correcto para nadie más que mi mismo, pero creo que puede ser un buen punto de partida para cualquier persona que quiera construir su máquina de trabajo desde cero, o simplemente quiera tener una referencia de cómo configurar ciertas herramientas y programas.

3 NixOS

El sistema operativo sobre el que construyo mi máquina de trabajo es Linux, más específicamente la distribución NixOS.

La razón por la que utilizo Linux es en realidad una lista extremadamente extensa que incluye razones tanto pragmáticas como inclusive filosóficas (fuck closed-source).

La razón por la que utilizo esta distribución es porque su estilo de configuración declarativo se lleva bastante bien con el flujo de documentación que

me gustaría llevar en este proyecto. El hecho de poder dejar de lado el aspecto imperativo de construir la máquina de trabajo y que en su lugar pueda declarar su estado final es algo muy bonito que veremos más adelante. Al final del día, uno debe escoger una distribución de Linux pensando siempre en la máxima que me dió un profesor de Administración de Sistemas Unix:

> "Todos los Linux son iguales"

Entonces, lo que cambia de una distribución a otra no es más que la comodidad con la que uno pueda trabajar con ella. Y en mi caso, me gusta mucho redactar especificaciones. El hecho de poder construir mi sistema a base de especificaciones me viene como anillo al dedo.

NixOS admite configuraciones reproducibles y declarativas a través de un manejo de configuraciones que se aplica a todo el sistema en todo momento.

Este sistema de configuraciones se basa en archivos .nix, que serán los que iremos construyendo para formar prácticamente todo el sistema.

3.1 Instalación y Sistema de Archivos

> Si BTRFS opina, estoy de acuerdo. Si BTRFS habla, lo escucho. Si BTRFS falla, mentira, el no falla. Si BTRFS piensa, lo avalo. Si BTRFS tiene 100 fans, yo soy uno de ellos. Si BTRFS tiene 1 fan, yo soy ese fan. Si BTRFS no tiene fans, es porque estoy muerto.

La base de mi máquina de trabajo será un sistema de archivos BTRFS con los siguientes subvolúmenes:

- Un subvolumen `/` el cual contendrá la raíz del sistema.
- Un subvolumen `/home` el cual contendrá el directorio `/home` del sistema
- Un subvolumen `/nix` el cual contendrá el directorio `/nix` del sistema

Las razones para separar mi sistema en tres subvolúmenes son un par y de varias índoles dependiendo de cada uno de estos subvolúmenes.

Para los tres porque me interesa tener políticas de snapshots distintas en cada uno. De la raíz, potencialmente quisiera guardar una snapshot en cada actualización del sistema sólo por si los ya existentes mecanismos de protección de NixOS no me pudieran salvar en algún momento; de `/home` me quiero poner paranóico y guardar snapshots de manera bastante regular, porque son mis archivos personales, y de `/nix` es totalmente innecesario tener

alguna política de snapshots, porque su contenido se genera de manera 100% declarativa.

Además, tener un subvolumen separado de `/home` me otorga el beneficio de independizar mis archivos personales de la instalación de Linux que tenga en un momento dado. Esto podría lograrlo de una manera incluso más fuerte con una partición separada, pero crear particiones trae el problema de eventualmente tener que redimensionarlas a nivel físico, y eso me da miedo.

Para instalar NixOS en BTRFS el proceso es bastante sencillo y directo, entonces omitiré esa sección y simplemente citaré este artículo de la Wiki al respecto.

Una vez instalado NixOS en la partición deseada, podemos empezar a configurar. Lo primero que si hay que hacer respecto a esto es importar las configuraciones de hardware que el instalador ha generado automáticamente, en un archivo llamado hardware-configuration.nix

```
{ inputs, config, lib, pkgs, ... }:

{
  imports =
    [ # Include the results of the hardware scan.
      /etc/nixos/hardware-configuration.nix
      <<imports>>
    ];
  fileSystems = {
  "/".options = [ "compress=zstd" ];
  "/home".options = [ "compress=zstd" ];
  "/nix".options = [ "compress=zstd" "noatime" ];
  };
```

De una vez también le declaramos al sistema operativo los subvolúmenes creados. Además, aquí hay una pequeña optimización respecto al subvolumen nix:

Cuando montamos una partición con la opción noatime, le prohibimos al sistema operativo actualizar el registro de "Access time".

Normalmente, cada vez que lees un archivo (aunque no lo modifiques), el sistema tiene que hacer una operación de escritura para actualizar la hora de acceso. Con noatime, esa escritura se elimina. Esto nos trae una optimización para el rendimiento y la vida útil de nuestro SSD que podemos tomar muy fácilmente aquí.

3.2 Configuraciones globales

El arranque de nuestro sistema será delegado a Systemd-boot. Esta es la primera vez que creo una configuración no basada en GRUB. Tal vez lo cambie más adelante, pero por ahora me gusta mucho el hecho de que este sistema "fuera de la caja" ya tenga soporte para las "Generaciones" de NixOS (lo veremos más adelante probablemente).

```
boot.loader.systemd-boot.enable = true;
boot.loader.efi.canTouchEfiVariables = true;
```

Y como está en el mejor de nuestros intereses de seguridad y features, siempre usaremos el kernel más reciente disponible

```
boot.kernelPackages = pkgs.linuxPackages;
```

El hostname de nuestra computadora es dependiente al contexto. Pero por ahora, mi máquina de trabajo es mi laptop.

```
networking.hostName = "mrtaichi-laptop";

networking.networkmanager.enable = true;
```

De una vez, desactivo el firewall. Esto porque generalmente, mi computadora personal suele encontrarse en situaciones en las cuales necesito desplegar servicios de manera rápida que queden a la escucha en alguno de mis puertos. Es un lío modificarlo sobre la marcha, así que prefiero cuidarme mucho.

```
networking.firewall.enable = false;
```

Luego para la zona horaria definimos que estamos en Ciudad de México.

```
time.timeZone = "America/Mexico_City";
```

Habilitamos el servidor X11. Aunque más adelante veremos que en realidad uso Hyprland (y por ende Wayland), todavía sirve tener esto por motivos de retrocompatibilidad. Hablando del servidor X11, también definimos aquí que use la distribución de teclado US-Intl con Altgr

```
services.xserver.enable = true;
services.xserver.xkb = {
layout = "us";
variant = "altgr-intl";
options = "grp:win_space_toggle";
};
```

Y activo el soporte para tener sonido en mi máquina de trabajo usando PipeWire.

```
services.pipewire = {
  enable = true;
  pulse.enable = true;
};
```

3.3 Cuentas de usuario

Mi equipo de trabajo solamente tiene un usuario interactivo: el mío.

No hay mucho que definir aquí excepto tal vez el preludio a 4 al definir que voy a usar zsh como emulador de terminal.

```
users.users.mrtaichi = {
  isNormalUser = true;
  extraGroups = [ "wheel" "libvirtd" <<userGroups>>]; # Enable ‘sudo’ for the user. 
  packages = with pkgs; [
    <<userPackages>>
  ];
  shell = pkgs.zsh;
};
```

Con esto terminamos las configuraciones iniciales respecto al sistema operativo como tal. El resto de configuraciones las haremos en la siguiente capa de abstracción: los programas :D

3.4 Utilidades principales

En NixOS la mayoría de los programas se pueden instalar de tres maneras: Las maneras declarativas son mediante la sección `environment.systemPackages`, donde podremos definir los programas que necesitaremos a nivel sistema; y mediante la sección `users.users.<nombre>.packages`. La diferencia entre

estas dos reside más que nada en la disponibilidad del programa. Los paquetes declarados como del sistema se instalan de manera global y al alcance de cualquier usuario, mientras que los paquetes instalados de manera local -o a nivel de usuario- justamente solo estáran disponibles para el usuario que se les haya definido. La norma es mantener los paquetes de sistema lo más reducidos posible, a solamente las cosas que nos hagan sentido que estén instaladas de manera global. La tercera forma es imperativa, mediante nix-shell. Esta forma no la voy a tratar aquí porque de hecho no la veremos en ninguna parte de nuestra configuración, pues está reservada para momentos en los que tenga que instalar programas de manera rápida y esté seguro de que los vaya a usar durante un periodo corto de tiempo.

Entonces, para definir los programas, herramientas y dependencias fijas de mi equipo de trabajo, a nivel sistema, lo haré de esta manera:

```
nixpkgs.config.allowUnfree = true;
 environment.systemPackages = with pkgs; [
   <<systemPackages>>
 ];
```

La primera línea es para permitir la instalación de software de código cerrado. Porque, aunque filosóficamente odio el código cerrado, he de admitir que hay ciertas instancias de mi flujo de trabajo que desgraciadamente dependen de algún software con copyright.

Tanto la sección de **systemPackages** como la de **userPackages** las seguiré tratando a lo largo del texto en varios flujos de trabajo (todos los que requieran instalación de programas, que son muchos) pero de una vez puedo empezar a declarar algunos esenciales.

3.4.1 System Packages

A nivel de sistema voy a definir los siguientes packages, comenzando por vim, wget, git y pre-commit

```
vim
wget
git
pre-commit
git-credential-oauth
```

Estos son mis esenciales para trabajar en un sistema mínimo, siendo vim mi editor de texto “fallback” para cuando necesite editar cosas desde

la consola de manera rápida (odio nano). Luego wget y git porque son esenciales para prácticamente cualquier dependencia necesaria en el sistema, y bueno, si hablamos de git, trabajar con este sistema de versiones es algo tan primordial para programadores que sobra la explicación. Pre-commit viene pegadita a git, ya que es uno de los hooks más populares para varios flujos de trabajo en git.

Luego tenemos alacritty, que es mi emulador de terminal preferido y el que me gustaría tener como emulador primordial y de "fallback"

```
alacritty
```

Bluetui, que es mi programa para gestionar mis dispositivos bluetooth desde la consola. Amo los programas TUI, desearía que el mundo se hubiera quedado en este stage y nunca hubieran nacido las GUIs.

```
bluetui
```

La Github CLI, para configurar mis credenciales de Github y poder subir cambios de manera rápida a mis repositorios ahí.

```
gh
```

Y finalmente la suite de TEX Live en su versión full, ya que compilo documentos en LATEX de manera bastante frecuente (de hecho siendo este trabajo un ejemplo bastante directo).

```
texlive.combined.scheme-full
```

3.4.2 User Packages

En el nivel de usuario voy a definir algunas herramientas generales que efectivamente usaré en varios flujos de trabajo, pero no considero que me den "vibras" de ser paquetes de sistema.

Empezamos con tree, que no es más que una herramienta de linea de comandos para mostrar de una manera más visual la estructura del directorio en el que estemos.

```
tree
```

También Spotify y Anki, que son mis herramientas de estudio esenciales. (sí, Spotify es mi herramienta de estudio favorita)

```
spotify
anki-bin
```

Y algunas utilidades básicas que necesitaré para cualquier flujo de trabajo en general, y que no caben en uno en específico (por ahora).

```
zip
file
remmina
htop
```

3.5 Flakes

Para algunos programas que vamos a usar más adelante, nos interesa mantener un registro claro de las versiones de las dependencias que estarán instalando y usando. Esto tanto para mantener la reproducibilidad como para ciertas situaciones nicho que requieren algunas versiones particulares de ciertos programas que vamos a utilizar.

Para esto, NixOS provee de una feature llamada Flakes, que en cierta forma hace esto. No voy a meterme en explicaciones porque a la fecha que escribo esto todavía no estoy tan versado en el mundo de Nix como para explicar claramente cómo funciona esto. Así que movámonos a lo pragmatico.

Para habilitar esta feature, agregamos lo siguiente a nuestro configuration.nix

```
nix.settings.experimental-features = [ "nix-command" "flakes" ];
```

Luego crearemos un archivo llamado flake.nix, el cual contendrá lo siguiente:

```
{
  description = "Configuración personal de Flakes";

  inputs = {
    nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";
    <<inputsFlakes>>
  };

  nixConfig = {
    extra-substituters = [
      <<substitutersFlakes>>
```

```
    ];
    extra-trusted-public-keys = [
      <<publicKeysFlakes>>
    ];
  };

  outputs = { self, nixpkgs, <<outputsFlakes>> ... }@inputs: {
    nixosConfigurations.mrtaichi-laptop = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        ./configuration.nix
        <<modulesFlakes>>
      ];
    };
  };
}
```

En este es en el que realizaremos los cambios necesarios para instalar paquetes de NixOS mediante Flakes. Generalmente solamente agregaremos cosillas a inputs, substituters o publicKeys. En casos particulares, también a modules, pero por default sólo contendría ./configuration.nix. Mismo caso con las outputs.

Además y para finalizar, ya que los packages que obtengamos mediante flakes generalmente no se encontrarán precompilados en la cache oficial de NixOS, varias veces tendremos que agregar las caches de los flakes que queramos agregar para ahorrarnos compilar todo el tiempo.

3.6 Cron

Cron es un programa de Linux que nos permite programar tareas para que se ejecuten automáticamente en momentos específicos. Es una herramienta muy útil para automatizar tareas repetitivas o programar tareas que deben ejecutarse en momentos específicos. Particularmente para este caso, voy a configurar ciertos cronjobs que me permiten mantener mi máquina de trabajo actualizada y con ciertas tareas de mantenimiento que me gusta hacer de manera regular.

Comenzamos por agregar el servicio de cron a nuestro configuration.nix

Y el primer cronjob que voy a declarar es que cada 2 semanas corra el garbage collector de NixOS, el cual se encarga de eliminar los paquetes y

dependencias que ya no se están utilizando en el sistema. Esto es algo que me gusta hacer de manera regular para mantener mi sistema limpio y evitar que se acumulen paquetes innecesarios.

```
  services.cron = {
    enable = true;
    systemCronJobs = [
      "0 0 */14 * *  root  nix-collect-garbage -d"
    ];
  };
```

3.7 SSH

Por motivos de mantenimiento (y para emergencias) siempre mantengo un acceso SSH en mis computadoras para cualquier necesidad que pueda surgir relacionada a lo que sea, desde recuperar algunos archivos que no tenga en la computadora en la que esté pero en otra sí, hasta algunos detalles ya más privados de cómo funciona mi flujo de trabajo de varios dispositivos.

Para habilitar esto en NixOS, empiezo definiendo que el servicio de SSH debe estar activo.

```
services.openssh = {
  enable = true;
  settings = {
    <<configuracionesSSH>>
  };
};
```

Luego, configuramos el servicio de la siguiente forma:

Primero, deshabilitamos la autenticación por contraseña. A todos mis equipos solo accederé mediante pares de llaves, nunca mediante contraseñas porque, aunque estoy bastante seguro de que mis contraseñas no aparecen en ninguna lista, no quiero tentar mi suerte. De la misma forma, desactivo los métodos de autenticación interactivos por teclado.

```
PasswordAuthentication = false;
KbdInteractiveAuthentication = false;
```

Luego, desactivamos el inicio de sesión como root, análogo a tema seguridad. Y hago explícito que sólo mi usuario puede autenticarse usando SSH.

```
PermitRootLogin = "no";
AllowUsers = [ "mrtaichi" ];
```

Y finalmente, agregamos penalizaciones para intentos de sesión fallidos. Es muy poco probable que una exposición de SSH mía salga fuera de una VPN o red local, pero por si acaso, no quiero ninguna sorpresa.

```
MaxAuthTries = 3;
PerSourcePenalties = "crash:3600s authfail:3600s max:86400s";
```

3.8 Configuraciones privadas

Una cosa bastante interesante de NixOS es que nos permite definir nuestra configuración a través de varios archivos .nix.

Esto se puede aprovechar de distintas maneras dependiendo de la forma que busques de configurar tu sistema. Tal vez quieras separar las configuraciones por temáticas para que cada .nix sea más legible, o según funcionalidades.

En mi caso, debido a que este documento ya sirve como una forma de darle legibilidad a mi configuración, lo único que haré para aprovechar esta feature será definir un private.nix, el cual será ignorado por este repositorio y contendrá configuraciones privadas que no quiero que queden aquí. Garantizo que ninguna es parte de mis flujos de trabajo.

Solo debemos de declarar en nuestros imports este archivo para que sea trackeado y usado a la hora de declarar nuestro sistema.

```
./private.nix
```

Y para que no lo trackee git, creamos su correspondiente .gitignore, y dentro del mismo excluimos el trackeo de este archivo.

```
private.nix
```

3.9 stateVersion

Antes de dejar de hablar de NixOS, debemos de abordar una variable muy importante que se define en estos archivos de configuración: system.stateVersion.

Esta variable guarda la primera versión de NixOS que le fue instalada al equipo. Esto con propósitos de mantener compatibilidad con los datos que no sean controlados por todo el mundo declarativo de NixOS. Esta variable entonces es inmutable, pero está altamente ligada al equipo físico en el que fué instalada mi máquina de trabajo y el momento en el que fue hecho esto.

Por este motivo, esta variable al igual que todas las configuraciones especificadas y generadas automáticamente en hardware-configuration.nix escapan al alcance de los archivos generados por este documento.

Por default esta variable vive en configuration.nix, que es el archivo de configuración que estamos generando en estos momentos, pero por lo explicado anteriormente, la moveré a hardware-configuration.nix, el cual es, como dije previamente, generado automáticamente por el instalador de NixOS cada vez que instale el sistema en un equipo físico nuevo.

4 OhMyZSH

El inicio de todo buen sistema es una buena shell. Es aquí donde la voy a pasar el 60% del tiempo porque mi flujo de trabajo está diseñado para depender lo menos posible de cualquier interfaz gráfica.

Para esto, me gusta mucho utilizar OhMyZSH, el cual, citando al mismo sitio oficial:

> Oh My Zsh is an open source, community-driven framework for managing your Zsh configuration.
>
> Sounds boring. Let's try again.
>
> Oh My Zsh will not make you a 10x developer... but you may feel like one!
>
> Once installed, your terminal shell will become the talk of the town or your money back! With each keystroke in your command prompt, you'll take advantage of the hundreds of powerful plugins and beautiful themes. Strangers will come up to you in cafés and ask you, "that is amazing! are you some sort of genius?"
>
> Finally, you'll begin to get the sort of attention that you have always felt you deserved. ... or maybe you'll use the time that you're saving to start flossing more often.

4.1 Instalación

En el archivo de configuración de NixOS agregamos las siguientes entradas:

```
programs.zsh = {
  enable = true; # Enable Zsh
ohMyZsh = {
  enable = true; # Enable Oh My Zsh
```

```
    plugins = [ "git" ]; # Add desired plugins
    # plugins = [ "git" "zsh-autosuggestions" "zsh-syntax-highlighting" ]; # Add desire
    theme = "agnoster"; # Set your preferred theme (default: robbyrussell)
    };
  };

  users.defaultUserShell = pkgs.zsh; # Set Zsh as the default shell
  environment.shells = [ pkgs.zsh ]; # Add Zsh to available shells
```

Y con eso tenemos :D

4.2 Configuraciones preliminares

Comenzaremos a ensamblar mi configuración para la shell de ZSH. En este capítulo vamos a abordar algunas configuraciones preliminares que no entran bajo ninguna categoría y en general solamente sirven para preparar el terreno para las features posteriores.

Primero comenzamos exportando una variable de entorno esencial para el funcionamiento de Oh My ZSH, la cual es la variable ZSH. Esta variable indica el path hacia la instalación de Oh My ZSH. Vamos a usar la ubicación por default.

```
export ZSH="$HOME/.oh-my-zsh"
```

Algunas configuraciones personales de mis dotfiles se encuentran en .profile, particularmente las relacionadas con variables de entorno apuntando a proyectos experimentales, o algunos alias rápidos que voy a usar durante una temporada determinada de mi vida pero que no valen la pena ser conservados. Es decir, configuraciones en general volátiles.

Zsh por defecto no sourcea .profile, ya que el estándar es que tenga su propio archivo llamado .Zprofile. Pero, por gusto propio, siempre he sido más de trabajar en .profile

```
[[ -e ~/.profile ]] && emulate sh -c 'source ~/.profile'
```

Luego definimos el tema de Oh My ZSH que voy a utilizar. A mi, particularmente, me gusta mucho el tema Agnoster.

```
ZSH_THEME="agnoster"
```

Si queremos cambiar de tema en un futuro, podemos hacerlo en base a los temas ya instalados en Oh My ZSH, los cuales podemos encontrar aquí.

Luego vamos a definir algunas cosillas como los plugins a utilizar. Oh My ZSH tiene una selección bastante amplia de plugins, pero yo soy más adepto a crear mis propias funcionalidades; por ahora solamente uso el de git porque me resulta bastante útil para saber en qué branch de un repositorio me encuentro en todo momento. De todas formas, tal vez en flujos de trabajo posteriores terminaré involucrando un plugin o dos.

```
plugins=(git <<pluginsOMZ>>)
```

Y, para finalizar con estas configuraciones iniciales, llamamos al script de inicialización de Oh My ZSH.

```
source $ZSH/oh-my-zsh.sh
```

4.3 Aliases

Un alias es, como su mismo nombre lo dice, un nombre corto para un comando largo que podemos definir usando el comando alias.

Tal vez debería de agregar más, pero por ahora los aliases que tengo son los siguientes:

4.3.1 Sudo

Se me hace tierno usar la palabra "pls" en lugar de sudo, lo cual además proporciona menos palabras, por lo que cumple con el propósito de ser un alias.

```
alias pls="sudo"
```

4.3.2 Arduino CLI

Cuando utilizo la CLI de Arduino siempre se me hace una tortura poner el nombre completo de la herramienta, así que lo reduzco a arcli

```
alias arcli="arduino-cli"
```

El resto de aliases las definiré por flujo de trabajo, usando la etiqueta noweb llamada aliasesCLI

```
<<aliasesCLI>>
```

4.4 PATH

PATH es una variable de entorno que se utiliza para indicarle a nuestra shell **las rutas del sistema donde puede encontrar archivos ejecutables**. Esto para que luego podamos simplemente correr estos programas por sus nombres en lugar de referirnos a rutas completas.

La variable PATH es una lista de rutas separada por :. A esta le vamos a agregar las rutas necesarias por nuestros programas para poder acceder a los mismos de manera más sencilla.

Comenzamos por agregar la ruta de los binarios de 5 la cual es por lo general:

```
PATH=$PATH:~/.config/emacs/bin
```

Luego la ruta de Flutter, que a veces suelo usar para interfaces gráficas sobre todo.

```
PATH=$PATH:/usr/bin/flutter/bin
```

Las del Android SDK, que uso sobre todo por adb y fastboot.

```
PATH=$PATH:$HOME/Android/Sdk/platform-tools
PATH=$PATH:$HOME/Android/Sdk/tools
```

De una vez también agrego la variable $\text{ANDROID}_{\text{HOME}}$, que es necesario para el Android SDK, y aunque no es directamente relacionado con esta sección, considero que tampoco es algo lo suficientemente grande como para abarcar con una sección propia.

```
ANDROID_HOME=$HOME/Android/Sdk
```

Generalmente me gusta usar la ruta ~/.local/bin para ejecutables variados que probablemente no quepan en alguna otra categoría y tampoco quiera agregar en las carpetas de ejecutables por default. Entonces también agrego esta ruta.

```
PATH=$PATH:$HOME/.local/bin
```

Luego también terminamos con Ruby y Golang, de los cuales de una vez voy también a definir la variable GEM_{HOME} que es necesaria para Ruby.

```
PATH=$PATH:$HOME/.local/share/gem/ruby/3.2.0/bin
PATH=$PATH:$HOME/go/bin
export GEM_HOME=$HOME/.gem
```

4.5 Terminal

Mi terminal predilecta es Alacritty, y para que i3 pueda invocar esta terminal adecuadamente, necesito fijar una variable de entorno TERMINAL.

```
export TERMINAL="alacritty"
```

5 Doom Emacs

Si todo este proyecto utiliza programación letrada, es imperativo que las primeras dotfiles que se creen sean las necesarias para hacer mi entorno de trabajo de programación letrada. Entonces comencemos con la herramienta principal, la cual es Doom Emacs. Si estás leyendo este documento en org, abajo verás el enlace al archivo org de toda esta herramienta, si estás leyendo PDF... pues sigue leyendo lol.

5.1 Introducción

Doom Emacs es mi principal editor de texto. Emacs en sí es el editor de texto que llevo usando desde que inicié la carrera -por influencia de la cultura de mi facultad-. El problema con Emacs es que es, a mi gusto, difícil de configurar en su versión básica. Por esto decidí usar Doom Emacs, el cual es un framework de configuración que me sirve para tener todo el poder de Emacs de una manera sencilla y que, además, no me hace perder la configurabilidad que me provee un Emacs vainilla.

5.2 Instalación

Para instalar Doom Emacs bajo NixOS lo único que necesitamos es tener la última versión de Emacs instalada en el equipo de trabajo. Para esto agregamos el paquete emacs a nuestros systemPackages.

```
emacs
```

Luego, ya que Doom Emacs es en realidad una colección de configuraciones que viven bajo el .config del usuario, habremos de instalarlo de manera imperativa, con las instrucciones tradicionales del repositorio:

```
git clone --depth 1 https://github.com/doomemacs/doomemacs ~/.config/emacs
~/.config/emacs/bin/doom install
```

Y listo.

Una vez hecho esto, solamente habremos de crear los enlaces simbólicos para importar las configuraciones que vamos a generar más adelante:

```
ln -sf ~/PersonalDotFiles/doomEmacs/config.el ~/.config/doom/config.el
ln -sf ~/PersonalDotFiles/doomEmacs/init.el ~/.config/doom/init.el
ln -sf ~/PersonalDotFiles/doomEmacs/packages.el ~/.config/doom/packages.el
doom sync
```

5.3 init.el

> Where you'll find your doom! block, which controls what Doom modules are enabled and in what order they will be loaded. This file is evaluated early when Emacs is starting up; before any other module has loaded. You generally shouldn't add code to this file unless you're targeting Doom's CLI or something that needs to be configured very early in the startup process.

En este archivo seleccionamos los *doom modules* que vamos a necesitar para el flujo de trabajo. Siendo la lista original la siguiente:

```
;;; init.el -*- lexical-binding: t; -*-

;; This file controls what Doom modules are enabled and what order they load
;; in. Remember to run 'doom sync' after modifying it!

;; NOTE Press 'SPC h d h' (or 'C-h d h' for non-vim users) to access Doom's
;;      documentation. There you'll find a link to Doom's Module Index where all
;;      of our modules are listed, including what flags they support.

;; NOTE Move your cursor over a module's name (or its flags) and press 'K' (or
;;      'C-c c k' for non-vim users) to view its documentation. This works on
;;      flags as well (those symbols that start with a plus).
;;
;;      Alternatively, press 'gd' (or 'C-c c d') on a module to browse its
;;      directory (for easy access to its source code).

(doom! :input
       ;;bidi              ; (tfel ot) thgir etirw uoy gnipleh
       ;;chinese
       ;;japanese
```

```
;;layout            ; auie,ctsrnm is the superior home row

:completion
;;(company +childframe)           ; the ultimate code completion backend
;;helm              ; the *other* search engine for love and life
;;ido               ; the other *other* search engine...
;;ivy               ; a search engine for love and life
;;vertico           ; the search engine of the future

:ui
;;deft              ; notational velocity for Emacs
;;doom              ; what makes DOOM look the way it does
;;doom-dashboard    ; a nifty splash screen for Emacs
;;doom-quit         ; DOOM quit-message prompts when you quit Emacs
;;(emoji +unicode)  ;
;;hl-todo           ; highlight TODO/FIXME/NOTE/DEPRECATED/HACK/REVIEW
;;hydra
;;indent-guides     ; highlighted indent columns
;;ligatures         ; ligatures and symbols to make your code pretty again
;;minimap           ; show a map of the code on the side
;;modeline          ; snazzy, Atom-inspired modeline, plus API
;;nav-flash         ; blink cursor line after big motions
;;neotree           ; a project drawer, like NERDTree for vim
;;ophints           ; highlight the region an operation acts on
;;(popup +defaults)   ; tame sudden yet inevitable temporary windows
;;tabs              ; a tab bar for Emacs
;;treemacs          ; a project drawer, like neotree but cooler
;;unicode           ; extended unicode support for various languages
;;(vc-gutter +pretty) ; vcs diff in the fringe
;;vi-tilde-fringe   ; fringe tildes to mark beyond EOB
;;window-select     ; visually switch windows
;;workspaces        ; tab emulation, persistence & separate workspaces
;;zen               ; distraction-free coding or writing

:editor
;;file-templates    ; auto-snippets for empty files
;;fold              ; (nigh) universal code folding
;;(format +onsave)  ; automated prettiness
;;god               ; run Emacs commands without modifier keys
;;lispy             ; vim for lisp, for people who don't like vim
```

```
;;multiple-cursors  ; editing in many places at once
;;objed             ; text object editing for the innocent
;;parinfer          ; turn lisp into python, sort of
;;rotate-text       ; cycle region at point between text candidates
;;snippets            ; my elves. They type so I don't have to
;;(evil +everywhere)
;;word-wrap         ; soft wrapping with language-aware indent

:emacs
;;dired             ; making dired pretty [functional]
;;electric          ; smarter, keyword-based electric-indent
;;ibuffer         ; interactive buffer management
;;undo              ; persistent, smarter undo for your inevitable mistakes
;;vc                ; version-control and Emacs, sitting in a tree

:term
;;eshell            ; the elisp shell that works everywhere
;;shell             ; simple shell REPL for Emacs
;;term              ; basic terminal emulator for Emacs
;;vterm             ; the best terminal emulation in Emacs

:checkers
;;syntax              ; tasing you for every semicolon you forget
;;(spell +flyspell) ; tasing you for misspelling mispelling
;;grammar           ; tasing grammar mistake every you make

:tools
;;ansible
;;biblio            ; Writes a PhD for you (citation needed)
;;collab            ; buffers with friends
;;debugger          ; FIXME stepping through code, to help you add bugs
;;direnv
;;docker
;;editorconfig      ; let someone else argue about tabs vs spaces
;;ein               ; tame Jupyter notebooks with emacs
;;(eval +overlay)     ; run code, run (also, repls)
;;lookup              ; navigate your code and its documentation
;;lsp               ; M-x vscode
;;magit             ; a git porcelain for Emacs
;;make              ; run make tasks from Emacs
```

```
;;pass              ; password manager for nerds
;;pdf               ; pdf enhancements
;;prodigy           ; FIXME managing external services & code builders
;;rgb               ; creating color strings
;;taskrunner        ; taskrunner for all your projects
;;terraform         ; infrastructure as code
;;tmux              ; an API for interacting with tmux
;;tree-sitter       ; syntax and parsing, sitting in a tree...
;;upload            ; map local to remote projects via ssh/ftp

:os
;;(:if (featurep :system 'macos) macos)  ; improve compatibility with macOS
;;tty               ; improve the terminal Emacs experience

:lang
;;agda              ; types of types of types of types...
;;beancount         ; mind the GAAP
;;(cc +lsp)         ; C > C++ == 1
;;clojure           ; java with a lisp
;;common-lisp       ; if you've seen one lisp, you've seen them all
;;coq               ; proofs-as-programs
;;crystal           ; ruby at the speed of c
;;csharp            ; unity, .NET, and mono shenanigans
;;data              ; config/data formats
;;(dart +flutter)   ; paint ui and not much else
;;dhall
;;elixir            ; erlang done right
;;elm               ; care for a cup of TEA?
;;emacs-lisp        ; drown in parentheses
;;erlang            ; an elegant language for a more civilized age
;;ess               ; emacs speaks statistics
;;factor
;;faust             ; dsp, but you get to keep your soul
;;fortran           ; in FORTRAN, GOD is REAL (unless declared INTEGER)
;;fsharp            ; ML stands for Microsoft's Language
;;fstar             ; (dependent) types and (monadic) effects and Z3
;;gdscript          ; the language you waited for
;;(go +lsp)         ; the hipster dialect
;;(graphql +lsp)    ; Give queries a REST
;;(haskell +lsp)    ; a language that's lazier than I am
```

```
;;hy                ; readability of scheme w/ speed of python
;;idris             ; a language you can depend on
;;json              ; At least it ain't XML
;;(java +lsp)       ; the poster child for carpal tunnel syndrome
;;javascript        ; all(hope(abandon(ye(who(enter(here))))))
;;julia             ; a better, faster MATLAB
;;kotlin            ; a better, slicker Java(Script)
;;latex             ; writing papers in Emacs has never been so fun
;;lean              ; for folks with too much to prove
;;ledger            ; be audit you can be
;;lua               ; one-based indices? one-based indices
;;markdown          ; writing docs for people to ignore
;;nim               ; python + lisp at the speed of c
;;nix               ; I hereby declare "nix geht mehr!"
;;ocaml             ; an objective camel
;;org               ; organize your plain life in plain text
;;php               ; perl's insecure younger brother
;;plantuml          ; diagrams for confusing people more
;;purescript        ; javascript, but functional
;;python            ; beautiful is better than ugly
;;qt                ; the 'cutest' gui framework ever
;;racket            ; a DSL for DSLs
;;raku              ; the artist formerly known as perl6
;;rest              ; Emacs as a REST client
;;rst               ; ReST in peace
;;(ruby +rails)     ; 1.step {|i| p "Ruby is #{i.even? ? 'love' : 'life'}"}
;;(rust +lsp)       ; Fe2O3.unwrap().unwrap().unwrap().unwrap()
;;scala             ; java, but good
;;(scheme +guile)   ; a fully conniving family of lisps
;;sh                ; she sells {ba,z,fi}sh shells on the C xor
;;sml
;;solidity          ; do you need a blockchain? No.
;;swift             ; who asked for emoji variables?
;;terra             ; Earth and Moon in alignment for performance.
;;web               ; the tubes
;;yaml              ; JSON, but readable
;;zig               ; C, but simpler

:email
;;(mu4e +org +gmail)
```

```
       ;;notmuch
       ;;(wanderlust +gmail)

       :app
       ;;calendar
       ;;emms
       ;;everywhere        ; *leave* Emacs!? You must be joking
       ;;irc               ; how neckbeards socialize
       ;;(rss +org)        ; emacs as an RSS reader
       ;;twitter           ; twitter client https://twitter.com/vnought

       :config
       ;;literate
       ;;(default +bindings +smartparens))
```

Para formar nuestra propia sección de init.el, iremos seleccionando los módulos necesarios. Considerando primero que nada que, todo lo que existe aquí es parte del bloque *doom!*. Entonces, nuestra primera declaración de este archivo debe de ser este bloque.

```
(doom!
    <<modulos_Elegidos>>
)
```

Luego, dentro de los módulos elegidos, necesitamos considerar que existen varias categorías de módulos para personalizar nuestra configuración de Doom Emacs, siendo las principales:

```
<<modulos_Input>>
<<modulos_Completion>>
<<modulos_Ui>>
<<modulos_Editor>>
<<modulos_Emacs>>
<<modulos_Term>>
<<modulos_Checkers>>
<<modulos_Tools>>
<<modulos_Os>>
<<modulos_Lang>>
```

```
<<modulos_Email>>
<<modulos_App>>
<<modulos_Config>>
```

Vamos a movernos entre cada categoría para armar nuestros selectos módulos.

5.3.1 Módulos Input

Esta categoría de módulos no posee una descripción oficial, y sus módulos tampoco están documentados, pero podemos asumir que son módulos que tienen que ver con soluciones de compatibilidad con layouts o idiomas con caracteres de entrada extraños (sea Chino o Japonés). Así que por el momento, no necesitamos nada de aquí.

```
:input
```

5.3.2 Módulos Completion

> Modules that provide new interfaces or frameworks for completion, including code completion.

Los dos módulos que incluye Doom Emacs en su configuración por defecto y que me han servido bastante hasta el momento han sido Company y vertico, la verdad es que no he probado los otros que ofrece, aunque no descarto hacerlo en un futuro.

```
:completion
(company +childframe)
vertico
```

5.3.3 Módulos UI

> Aesthetic modules that affect the Emacs interface or user experience.

En esta sección también tengo por ahora una configuración directamente default, aunque al estar escribiendo este documento, me pongo a leer las descripciones de los módulos que puedo elegir para mejorar mi interfaz de usuario, y me tientan. Tal vez otro día me pongo a seleccionarlos como se debe.

```
:ui
doom
doom-dashboard
hl-todo
modeline
ophints
(popup + defaults)
(vc-gutter +pretty)
vi-tilde-fringe
workspaces
```

5.3.4 Módulos Editor

> Modules that affect and augment your ability to manipulate or insert text.

Aquí ya le agregamos un poco más de salsa al asunto, porque esta es la parte donde decido confesar que llevo un año o más usando Emacs en evil mode. Porque -al César lo que es del César- la verdad es que me encantan los atajos de Vim, a lo largo de estos últimos años he intentado integrarlos a cualquier parte de mi flujo de trabajo que se dejara. Y no sé si sentirme orgulloso de decir que, es muy probable que una persona que no sepa usar Vim no sepa usar mis computadoras.

```
:editor
file-templates
fold
snippets
(evil +everywhere)
```

5.3.5 Módulos Emacs

> Modules that reconfigure or augment packages or features built into Emacs.

En esta categoría también tenemos los defaults por ahora.

```
:emacs
dired
electric
```

```
undo
vc
```

5.3.6 Módulos Term

> Modules that offer terminal emulation.

Para esta categoría vamos a incluir el módulo de vterm, que presumiblemente es el mejor emulador de terminal hasta el momento que tenemos disponible para Doom Emacs.

```
:term
vterm
```

5.3.7 Módulos Checkers

Esta categoría de módulos no tiene una descripción oficial, pero podemos decir que engloba aquellos módulos relacionados con la corrección de ortografía.

Y pues por ahora solamente utilizo el verificador de syntax.

```
:checkers
syntax
```

5.3.8 Módulos Tools

> Small modules that give Emacs access to external tools & services.

Dentro de esta categoría incluiremos los siguientes módulos:

1. Eval Para realizar evaluaciones de bloques de código en el momento. Me parece que este es uno de los módulos habilitados por default, así que desconozco realmente la funcionalidad de tenerlo habilitado.

2. Lookup Para navegar a través del código, buscando "símbolos" y definiciones.

3. Lsp Mi favorito, con este podemos tener servidores de lenguaje, que nos implementan varias de las funcionalidades que podríamos extrañar de

un IDE convencional. Con este módulo, junto con los correspondientes a cada lenguaje que necesitemos (en Módulos Lenguajes), convertimos a Emacs en un IDE completo.

4. Magit Agrega integración con Git, principalmente lo uso para ver en el lateral izquierdo los cambios que he introducido a un archivo y que no he commiteado.

5. PDF Agrega mejoras para poder trabajar con PDFs dentro de Emacs. Esto me resulta muy útil para poder trabajar con LaTeX y org-mode en una ventana mientras en la otra veo la exportación final en PDF, verificando que se vea de la forma que yo quiero.

   Finalmente, esta sección queda de la siguiente forma:

```
:tools
lookup
lsp
magit
pdf
```

5.3.9 Módulos OS

> Modules to improve integration into your OS, system, or devices.

El módulo que viene por default es el de MacOS, bajo el condicional de activarse solamente si detecta que estamos bajo dicho sistema operativo. Lo dejo activo, solamente porque así, si un día cambio a MacOS, no me ocurra que me confunda porque no funciona y me de cuenta que fué por desactivar este módulo.

```
:os
(:if (featurep :system 'macos) macos)
```

5.3.10 Módulos de lenguajes

> Modules that bring support for a language or group of languages to Emacs.

Esta es una de las categorías más importantes para un programador, pues literalmente aquí podemos encontrar los paquetes de soporte para lenguajes de programación que nos proporciona Emacs. Me encantaría poder activar todos de una vez, porque sé que al ritmo que voy eventualmente los terminaría usando; pero la traba es que mientras más módulos tenga activos en esta setup, me exhibo más a tiempos de arranque reducidos y posibles pérdidas de rendimiento. Por este motivo, prefiero solamente activar estos módulos conforme los vaya necesitando. Declaro la etiqueta $\text{modulos}_{\text{Lang}}$, y será la que usaré para agregar módulos a esta sección.

```
:lang
emacs-lisp
json
latex
markdown
(rust +lsp)
sh
(org +evil +dragndrop +pretty)
```

El intended debería de ser que cada lenguaje de programación tenga su propia sección en 11, y con ello, su propio flujo de trabajo curado y asentado. Pero por ahora, dejaré varios lenguajes aquí sueltos sólo para no perder capacidades en mi IDE, mientras creo los flujos de trabajo correspondientes para cada idioma.

Algo para comentar particularmente aquí es que Emacs tiene soporte para org-mode de manera nativa, entonces uno podría pensar que no es necesario el módulo de org. La principal razón por la que lo agrego es por compatibilidad con Evil Mode. Si no explicito esta compatibilidad, los bindings dentro de la Org-Agenda se revierten a los defaults y eso me hace mucha fricción para trabajar... Además, agrego soporte para drag and drop y la flag de pretty para que se vea más bonito :)

5.3.11 Módulos de Email

Esta categoría no posee una descripción oficial, pero proporciona los medios para poder conectar un cliente de correo a Emacs.

Me parece un campo sin dudar bastante atractivo poder usar mi correo electrónico dentro de Emacs. Pero por ahora creo que prefiero quedarme así, tal vez en un futuro decida utilizar esta posible feature.

```
:email
```

5.3.12 Módulos de Apps

> Application modules are complex and opinionated modules that transform Emacs toward a specific purpose. They may have additional dependencies and should be loaded last, before :config modules.

Por ahora, sólo utilizo emacs-everywhere, que me permite abrir frames de Emacs para editar cualquier recuadro de texto en mi computadora usando Emacs. Esto es algo bastante útil que no uso tan seguido como podría hacerlo, pero definitivamente cada que lo uso ~~siento como pierdo sex appeal~~ me siento como un power user.

```
:app
everywhere
```

5.3.13 Módulos de configuración

> Modules that configure Emacs one way or another, or focus on making it easier for you to customize it yourself. It is best to load these last.

Nada interesante aquí, excepto señalar que esta categoría contiene dentro de sí al módulo literate. Este módulo nos permite tanglear automáticamente un archivo config.org al iniciar nuestro Emacs. Esto con el propósito de que este config.org funcione como nuestra configuración letrada para Doom Emacs.

Podría haber usado este módulo para tanglear la configuración de Doom Emacs particularmente, pero siempre tuve pensado este archivo .org como algo que comprendiera completamente a mis dotfiles, como una especie de libro maestro para mi flujo de trabajo. Entonces, tener mi configuración de Doom Emacs viviendo en otro archivo .org me hubiera resultado conflictivo. Y evidentemente usar este config.org para el libro maestro y así tanglear todas mis dotfiles cada que abra Emacs tampoco hubiera sido una opción a considerar...

Así que nos quedamos solamente con el módulo default, el cual **obviamente** viene activado por default.

```
:config
(default +bindings +smartparens)
```

5.4 config.el

> Here is where 99.99% of your private configuration should go. Anything in here is evaluated after all other modules have loaded, when starting up Emacs.

Esta es la parte más sabrosa de la configuración de Doom Emacs en cuanto a usabilidad se trata. Una vez hemos elegido los módulos que vamos a utilizar dentro de nuestro *init.el*, podemos empezar a ajustar nuestras configuraciones.

Vamos a dividir la estructura de este archivo por las features a las que aportan las configuraciones necesarias. La estructura actual se verá de la siguiente forma, separando la configuración por features:

```
<<esteticaYEstilo>>
<<orgMode>>
<<atomicChrome>>
<<latex>>
<<programacionLetrada>>
```

5.4.1 Estética y estilo

Primero comenzamos con la elección de tema, la cual hacemos cambiando el nombre de la variable *doom-theme*. No recuerdo bien si he cambiado este tema o el que estoy usando es el default, pero en todo caso, actualmente utilizo doom-one, y no he tenido ninguna queja hasta el momento.

```
(setq doom-theme 'doom-one)
```

Luego definimos la variable display-line-numbers-type, la cual nos permite controlar como se va a mostrar la barra lateral que nos indica la línea del documento sobre la que estamos. Existen varias configuraciones posibles, incluso para hacerla relativa. Yo prefiero quedarme con la sencilla, la cual simplemente marca en el lado izquierdo la línea en la que nos hallamos.

```
(setq display-line-numbers-type t)
```

Luego está el tema de las transparencias. A mí me gusta definir una ligera transparencia en el marco principal de Emacs, porque esto me permite poder ver ligeramente la ventana que tenga acomodada detrás de la ventana de Emacs. Esto me deja tener Emacs en pantalla completa y tener atrás algún documento para consulta, lo cual me ha resultado útil en muchos casos.

```
(set-frame-parameter (selected-frame) 'alpha '(95 95))
(add-to-list 'default-frame-alist '(alpha 95 95))
```

La transparencia la tengo definida a 95, pues es el valor que considero lo suficientemente opaco para no perder el texto de la ventana de Emacs, pero lo suficientemente transparente para poder ver texto detrás.

5.4.2 Org Mode

Para poder tener un espacio de trabajo adecuado a mis necesidades, necesito realizar ciertas configuraciones a los plugins de mi Emacs relacionados con Org-Mode. Debo de admitir que, por ahora, mi setup de trabajo de org-mode, aunque funcional, sigue siendo bastante rudimentaria a comparación de varios sistemas que he visto en Internet.

En general solamente uso algunas cosillas básicas como org-agenda y babel para mis necesidades de literate programming. Comencemos con org-agenda.

La variable org-directory define la ruta por defecto en la que vamos a guardar todos los archivos .org que pueda necesitar para mi flujo de trabajo. Esto NO es lo mismo que la variable org-agenda-files, la cual explicaré en su momento. Por ahora no uso mucho este directorio, pero nunca está de más tener su definición clara para más adelante.

En un futuro, aquí podría colocar notas, planificación, gestión de proyectos, y en general los archivos auxiliares que necesite cuando enriquezca mi flujo de trabajo GTD.

```
(setq org-directory "~/org/")
```

Luego defino org-agenda-files, la cual es la variable que org-agenda utiliza para encontrar los archivos .org de los que va a construir la agenda. Es decir, de los archivos .org que vivan dentro de este directorio es donde la Agenda va a extraer los TODOs, tareas, proyectos, deadlines, etc que tenga en el momento y me los va a mostrar en la Org Agenda View.

```
(setq org-agenda-files '("~/Documents/agendaORG"))
```

Tal vez luego debería de documentar también mis flujos de trabajo, pero eso por ahora se escapa al enfoque de este proyecto.

Algo que necesito que org-mode respete para mi agenda son los siguientes puntos:

- Me gusta planificar mi semana con antelación, y para ello me gusta siempre tener claro cuáles de mis tareas con Deadline próxima ya fueron consideradas en esta planeación y cuáles no. Quiero olvidarme de las que ya fueron Scheduled hasta que llegue el día que planee para hacerlas, por desgracia, org-mode gusta de mostrar siempre las Deadlines próximas, sin importarle esto, lo cual me estresa. Para esto, configuro Emacs para que solamente me avise de las tareas que tienen una Deadline proxíma pero no hayan sido Scheduleds, pues esto significa que no las he considerado dentro de mi planeación.

  ```
  (setq org-agenda-skip-deadline-prewarning-if-scheduled 'pre-scheduled)
  ```

- Mi método de planificación es semanal, es decir, cada domingo me siento a revisar todos mis archivos de orgAgenda para revisar qué pendientes necesito planificar y construyo mi semana en base a eso. Particularmente me interesa darle énfasis a los pendientes cuya deadline caiga dentro de la semana y no haya planeado, me gusta entonces que org-mode me avise de estos. Pero también me gusta ir pensando en los pendientes cuya deadline se aproxima a la siguiente semana; solamente en caso de que sea necesario comenzar a trabajarlos con dos semanas de antelación. Para esto defino la variable org-deadline-warning-days como 14, esto combinado con la variable pasada me permite tener una visión más clara de mis pendientes planeados, no planeados, y particularmente de los pendientes cuya deadline es próxima y no he planeado.

  ```
  (setq org-deadline-warning-days 14)
  ```

- Otra cosa que uso para eliminar ruido de la vista de Agenda es asegurarme de que los pendientes que ya tengo hechos (marcados como DONE) desaparezcan de mi vista, esto me ayuda a concentrarme en los que no he terminado. Para esto cambio tres variables.

  ```
  (setq org-agenda-skip-scheduled-if-done t
        org-agenda-skip-deadline-if-done t
        org-agenda-skip-timestamp-if-done t)
  ```

- Finalmente, también agrego una configuración que hace que, al abrir los archivos `.org`, sus títulos y subtítulos aparezcan primero con sus contenidos colapsados. De esta manera puedo moverme entre ellos de manera mucho más rápida que si viera todos los títulos y su contenido de un solo vistazo de buenas a primeras.

```
(after! org
  (setq org-startup-folded t))
```

Estas son las especificaciones principales de mi flujo de trabajo, luego solamente agrego un pequeño detalle de calidad de vida para poder usar atajos de teclado para cambiar los estados de mis tareas rápidamente.

```
(setq org-use-fast-todo-selection t)
```

Y finalmente, los estados que utilizo para mis tareas. Estos están directamente inspirados del flujo de trabajo de otra persona, pero no recuerdo quién jajaja. Tal vez cuando haga mi documentación sobre mi flujo de trabajo lo defina de manera más clara.

```
(setq org-todo-keywords
   '((sequence "TODO(t)" "NEXT(n)" "PROJ(p)" "|" "DONE(d)")
  (sequence "TASK(T)")
  (sequence "WAITING(w@/!)" "INACTIVE(i)" "SOMEDAY(s)" "|" "CANCELLED(c@/!)")))

(setq org-todo-keyword-faces
      '(("TODO" :foreground "red" :weight bold)
     ("TASK" :foreground "#5C888B" :weight bold)
     ("NEXT" :foreground "blue" :weight bold)
     ("PROJ" :foreground "magenta" :weight bold)
     ("DONE" :foreground "forest green" :weight bold)
     ("WAITING" :foreground "orange" :weight bold)
     ("INACTIVE" :foreground "magenta" :weight bold)
     ("SOMEDAY" :foreground "cyan" :weight bold)
     ("CANCELLED" :foreground "forest green" :weight bold)))
```

De una vez también defino los colores para cada estado.

- Org-agenda crea algo llamado "Logbook" para tareas que se completan de manera diaria. Esto en resúmen lo que hace es crear una lista de veces que se ha completado una tarea bajo el título de la misma. El problema es que para tareas que se completan todos los días (hábitos por ejemplo) esto crea listas muy largas, de este estilo:

  ```
   ** TODO Revisar correo electrónico
  SCHEDULED: <2026-01-12 Mon +1d>
  ```

```
:PROPERTIES:
:LAST_REPEAT: [2026-01-11 Sun 17:49]
:END:
- State "DONE"       from "TODO"       [2026-01-11 Sun 17:49]
- State "DONE"       from "TODO"       [2026-01-11 Sun 17:49]
- State "DONE"       from "TODO"       [2026-01-09 Fri 17:47]
- State "DONE"       from "TODO"       [2026-01-08 Thu 21:41]
- State "DONE"       from "TODO"       [2026-01-08 Thu 14:31]
- State "DONE"       from "TODO"       [2026-01-06 Tue 11:41]
- State "DONE"       from "TODO"       [2026-01-05 Mon 19:54]
- State "DONE"       from "TODO"       [2026-01-04 Sun 21:48]
- State "DONE"       from "TODO"       [2026-01-03 Sat 19:35]
- State "DONE"       from "TODO"       [2026-01-02 Fri 19:54]
- State "DONE"       from "TODO"       [2026-01-01 Thu 23:36]
- State "DONE"       from "TODO"       [2026-01-01 Thu 23:36]
- State "DONE"       from "TODO"       [2025-12-30 Tue 15:28]
- State "DONE"       from "TODO"       [2025-10-06 Mon 18:11]
```

Para aminorar esto, la siguiente linea hace que todas las lineas del logbook se guarden en un drawer del mismo nombre abajo de la tarea correspondiente.

```
(setq org-log-into-drawer t)
```

Finalmente, agregamos las líneas correspondientes a activar el `package` Org Edna. La función de este paquete la explico en la sección de 5.5

```
(after! org
  (require 'org-edna)
  (org-edna-mode 1))
```

5.4.3 Atomic Chrome

Esta es una configuración que viene de tiempos en los que me hice directamente dependiente de los atajos de Vim y me sentía un completo inútil en entornos donde no los pudiera usar (me sigue pasando, pero ahora uso Vimium C). Particularmente, para editar texto en el navegador me gustaba poder abrir un buffer de Emacs, editar el texto ahí y regresar a la aburrida vida del navegador. Esto ha ido cayendo en desuso por mi flujo de trabajo debido a que poco a poco he usado cosas como Vimium C y me he vuelto a acostumbrar a usar el mouse, pero conservo estas configuraciones por si algún día las necesito.

> This is the Emacs version of Atomic Chrome which is an extension for Google Chrome browser that allows you to edit text areas of the browser in Emacs. It's similar to Edit with Emacs, but has some advantages as below with the help of websocket.

Para esto, no se necesita más que requerir (obviamente) el susodicho plugin Atomic Chrome y habilitar el servidor cada vez que abra emacs.

```
(require 'atomic-chrome)
(atomic-chrome-start-server)
```

5.4.4 LaTeX

Por ahora no realizo otra configuración que no sea ajustar mi visor preferido para mis compilados de LaTeX, el cual es KDE Okular, porque honestamente es una chulada de visor.

```
(setq +latex-viewers '(okular))
```

5.4.5 Programación Letrada

Mis configuraciones relacionadas con Literate Programming y los documentos letrados que genero con este precioso paradigma de programación (o marco de trabajo?)

Por ahora tengo un flujo de trabajo bastante sencillo, por lo que solamente defino dos cosas:

- Los lenguajes de programación que uso para mis documentos letrados, esta lista es extremadamente mutable conforme van creciendo mis proyectos.

  ```
  (org-babel-do-load-languages
  'org-babel-load-languages
  '((emacs-lisp . t)
    (go . t)
    (haskell . t)
    (sql . t)
    ))
  ```

- Me gusta ponerle imágenes a mis proyectos (no se nota en este lol) y me gusta que se vean no solamente en sus versiones imprimibles, pero también en el mismo editor.

```
(setq org-startup-with-inline-images t)
```

Con esto terminamos la subsección dedicada a mi config.el, ya casi nos falta poco para terminar la sección de Doom Emacs, solamente nos falta la sección de packages.el

5.5 packages.el

> This file is where package declarations belong. It's also a good place to look if you want to see what packages a module manages (and where they are installed from).

Aquí, según la documentación oficial de Doom Emacs, definiremos los packages que vamos a usar y que no se encuentran en alguno de los módulos definidos previamente. Primero agregamos el paquete `atomic-chrome`, el cual nos servirá para complementar el flujo de trabajo entre Emacs y navegador que describo en 5.4

```
(package! atomic-chrome)
```

Parte del flujo de trabajo -que debería de documentar- en mi agenda Org requiere establecer relaciones de dependencia entre tareas que puede que no se encuentren en el mismo árbol, o tal vez incluso, ni siquiera en el mismo archivo. Para esto, necesito agregar el `package` Org Edna, el cual nos permite justamente realizar esto.

```
(package! org-edna)
```

Terminando este archivo, finalmente tenemos completa la configuración de Doom Emacs

5.6 Pegar imágenes del portapapeles

Una parte necesaria de mi flujo de trabajo dentro de Emacs, sobre todo para elaboración de reportes y tareas, es la capacidad de poder pegar imágenes directamente dentro de los documentos `.org`. Particularmente, capturas de pantalla que tenga guardadas en el portapapeles.

Para esto, voy a usar un paquete llamado `org-download` que, acompañado con algunos tweaks, cumple exactamente con esta función.

Primero agregamos el paquete a `packages.el`

```
(package! org-download)
```

Y agregamos la configuración necesaria a mi `config.el`

```
(after! org-download
    (setq org-download-method 'directory)
    (setq org-download-image-dir (concat (file-name-sans-extension (buffer-file-name))
    (setq org-download-image-org-width 600)
    (setq org-download-link-format "[[file:%s]]\n"
          org-download-abbreviate-filename-function #'file-relative-name)
    (setq org-download-link-format-function #'org-download-link-format-function-defaul
```

Estas lineas de configuración solamente hacen estos ajustes:

- `(after!  org-download ...)`

Nos asegura que estas configuraciones se apliquen únicamente después de que el paquete org-download haya sido cargado, evitando errores al iniciar Emacs.

- `(setq org-download-method 'directory)`

Establece que las imágenes guardadas se colocarán en un directorio físico dentro de tu sistema de archivos (en lugar de otros métodos de almacenamiento).

- `(setq org-download-image-dir ...)`

Define la ruta de la carpeta donde se guardarán las imágenes de forma automática:

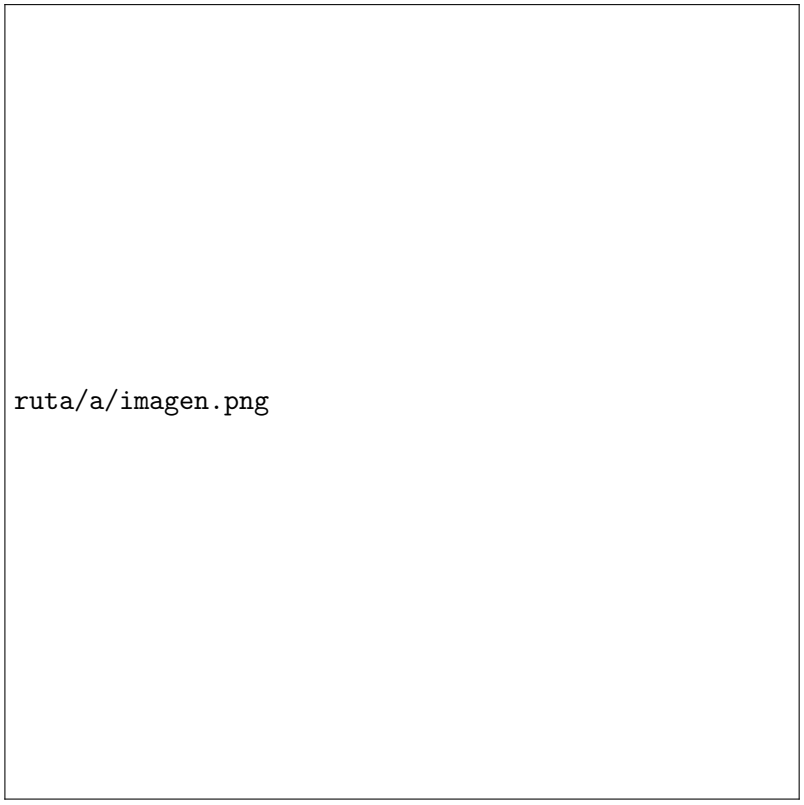
Toma el nombre de tu archivo de notas actual y le quita la extensión (file-name-sans-extension). Le añade el sufijo -img.

Ejemplo: Si tu documento se llama notas.org, se creará automáticamente una carpeta llamada notas-img/ en la misma ruta para almacenar todas las imágenes de ese archivo.

- `(setq org-download-image-org-width 600)`

Le indica a Org-mode que reescale visualmente la imagen a un ancho de 600 píxeles dentro de Emacs (insertando el atributo #+ATTR_{ORG}: :width 600 en el enlace).

- `(setq org-download-link-format "[[file:%s]]\n" org-download-abbreviate-filename-fu #'file-relative-name)` org-download-link-format: Define la sintaxis de Org-mode para insertar la imagen (



). org-download-abbreviate-filename-function #'file-relative-name: Fuerza a que la ruta de la imagen sea relativa al archivo .org en lugar de guardar una ruta absoluta (ej. notas-img/foto.png en vez de /home/usuario/documentos/notas-img/foto.png). Esto permite mover la carpeta de tus notas sin romper las imágenes.

- `(setq org-download-link-format-function ...)`

Restaura o asegura el uso de la función por defecto para dar formato al enlace de la imagen, garantizando compatibilidad con las variables configuradas en el paso anterior.

Pero, para que todo esto funcione, necesito agregar una dependencia más a mi sistema. Esta es wl-clipboard.

wl-clipboard es un paquete del sistema que permite, como su nombre lo dice, manipular el portapapeles del sistema desde la linea de comandos. Es necesario porque, es justo lo que necesita hacer toda esta parte de mi flujo de trabajo para funcionar. Lo agregamos a los paquetes de sistema.

```
wl-clipboard
```

6 Hyprland

Desde que conozco los gestores de ventanas y la gran ventaja de rapidez que ofrecen cuando les tomas el ritmo correcto (porque es inherentemente más rápido manejar la computadora con el teclado que con el mouse), no he logrado escapar de usar uno en niguna de las encarnaciones de mis estaciones de trabajo. Si eres lector de mi blog, puedes consultar un artículo al respecto, que de hecho es la primera publicación de mi blog.

A tal punto de que lo primero que suelo hacer cuando llega una nueva computadora a mis manos es reemplazar su entorno de escritorio por un gestor de ventanas. Esto es algo bastante fácil de realizar en Linux (que es el enfoque de estas dotfiles), y un poco más complicado de realizar en otros sistemas operativos, pero no imposible.

Previamente usaba un entorno de escritorio basado en KDE Plasma al cual le reemplazaba su gestor de ventanas nativo (KWin) por i3. Esto me permitía conservar todo lo mejor de ambos mundos: la integración y herramientas que te ofrece tener un entorno de escritorio completo con su respectivo ecosistema curado por alguna comunidad dedicada a eso; junto con la sencilleza y agilidad que provee el manejar tus ventanas con puros atajos de teclado y acomodados automáticos de un gestor de ventanas.

Pero desde los cambios que ha hecho este ecosistema para migrar a Wayland, esta alternativa se ha vuelto cada vez más inviable en cada actualización que pasa de KDE, llegando a un punto en el que directamente no pude hacerlo funcionar. Esto se debe a la naturaleza del nuevo KDE Wayland, donde directamente es imposible (al menos a la fecha) de sustituir su gestor de ventanas nativo.

Esto me lleva a escribir esta nueva configuración: intentar abrazar de manera incómoda el avance del mundo y utilizar un gestor de ventanas moderno: Hyprland. Todavía tengo un gusto agridulce al respecto de esto, pero ni modo, vamos a comenzar.

6.1 Instalación

Vamos a utilizar la versión de desarrollo de Hyprland, porque es la única que admite plugins que vamos a utilizar más adelante. Así que necesitaremos configurar un Flake dentro de nuestro flakes.nix:

```
hyprland = {
```

```
  url = "github:hyprwm/Hyprland/v0.56.0";
};
```

Para ahorrarnos compilaciones innecesarias, también agregamos el repositorio cache de Hyprland. Primero lo declaramos en substituters:

```
"https://hyprland.cachix.org"
```

Y damos la llave pública del mismo:

```
"hyprland.cachix.org-1:a7pgxzMz7+chwVL3/pzj6jIBMioiJM7ypFP8PwtkuGc="
```

Y nos movemos a configuration.nix para comenzar a configurar el programa como tal. Para utilizar la versión de Hyprland de Flakes que hemos definido, debemos agregar esto a nuestros imports:

```
inputs.hyprland.nixosModules.default
```

Declaramos las siguientes lineas en el configuration.nix:

```
  services.displayManager.sddm.enable = true;
  services.desktopManager.plasma6.enable = true;

  programs.hyprland = {
    enable = true;
    package = pkgs.hyprland;
    # make sure to also set the portal package, so that they are in sync
    portalPackage = pkgs.xdg-desktop-portal-hyprland;
    # usual Nixpkgs module options
    plugins = [
      <<pluginsHyprland>>
    ];
    settings = {
      <<configuracionHyprland>>
    };
  };

security.wrappers.Hyprland.enable = false;
```

Para las dependencias que especificaremos dentro de systemPackages, tendremos las siguientes:

```
rofi
hyprlock
hyprshot
hyprpaper
waybar
```

6.2 Hy3

Hy3 es un plugin para Hyprland que me permite tener stacking, algo que en Hyprland nativo no existe. Sin stacking terminaría teniendo 6 espacios de trabajo en mi flujo de trabajo más tranquilo, con stacking, podría reducirlo a sólo dos espacios. Es evidente entonces que es algo esencial.

Y declaramos su uso en la configuración de plugins de Hyprland.

```
pkgs.hyprlandPlugins.hy3
```

6.3 Configuraciones (nuevo)

TODO: Documentar este pedo

```
"plugin:hy3" = {};

  # --- AUTOSTART ---
  exec-once = [
    "hyprpaper"
    "waybar"
    "dbus-update-activation-environment --systemd WAYLAND_DISPLAY XDG_CURRENT_DESKTOP"
    "dropbox start"
    "gnome-keyring-daemon --start --components=secrets,pkcs11,ssh"
    # Carga del plugin hy3
    "hyprctl plugin load ${pkgs.hyprlandPlugins.hy3}/lib/libhy3.so"
  ];

  extraConfig = ''

# See https://wiki.hypr.land/Configuring/Monitors/
monitor= HDMI-A-1,preferred,auto-up,1, mirror, eDP-1
monitor=,highres,auto,auto


###################
```

```
### MY PROGRAMS ###
####################

# See https://wiki.hypr.land/Configuring/Keywords/

# Set programs that you use
$terminal = alacritty
$menu = ~/.config/rofi/launchers/type-1/launcher.sh

env = XCURSOR_SIZE,24
env = HYPRCURSOR_SIZE,24


####################
### PERMISSIONS ###
####################

# See https://wiki.hypr.land/Configuring/Permissions/
# Please note permission changes here require a Hyprland restart and are not applied on
# for security reasons

# ecosystem {
#   enforce_permissions = 1
# }

# permission = /usr/(bin|local/bin)/grim, screencopy, allow
# permission = /usr/(lib|libexec|lib64)/xdg-desktop-portal-hyprland, screencopy, allow
# permission = /usr/(bin|local/bin)/hyprpm, plugin, allow


#####################
### LOOK AND FEEL ###
#####################

# Refer to https://wiki.hypr.land/Configuring/Variables/

# https://wiki.hypr.land/Configuring/Variables/#general
general {
    gaps_in = 3
    gaps_out = 5
```

```
    border_size = 1

    # https://wiki.hypr.land/Configuring/Variables/#variable-types for info about colo
    col.active_border = rgba(33ccffee) rgba(00ff99ee) 45deg
    col.inactive_border = rgba(595959aa)

    # Set to true enable resizing windows by clicking and dragging on borders and gaps
    resize_on_border = false

    # Please see https://wiki.hypr.land/Configuring/Tearing/ before you turn this on
    allow_tearing = false

    layout = hy3
}

# https://wiki.hypr.land/Configuring/Variables/#decoration
decoration {
    rounding = 10
    rounding_power = 2

    # Change transparency of focused and unfocused windows
    active_opacity = 1.0
    inactive_opacity = 1.0

    shadow {
        enabled = true
        range = 4
        render_power = 3
        color = rgba(1a1a1aee)
    }

    # https://wiki.hypr.land/Configuring/Variables/#blur
    blur {
        enabled = true
        size = 3
        passes = 1

        vibrancy = 0.1696
    }
```

```
}

# https://wiki.hypr.land/Configuring/Variables/#animations
animations {
    enabled = yes, please :)

    # Default curves, see https://wiki.hypr.land/Configuring/Animations/#curves
    #        NAME,           X0,   Y0,   X1,   Y1
    bezier = easeOutQuint,   0.23, 1,    0.32, 1
    bezier = easeInOutCubic, 0.65, 0.05, 0.36, 1
    bezier = linear,         0,    0,    1,    1
    bezier = almostLinear,   0.5,  0.5,  0.75, 1
    bezier = quick,          0.15, 0,    0.1,  1

    # Default animations, see https://wiki.hypr.land/Configuring/Animations/
    #           NAME,          ONOFF, SPEED, CURVE,        [STYLE]
    animation = global,        1,     10,    default
    animation = border,        1,     5.39,  easeOutQuint
    animation = windows,       1,     4.79,  easeOutQuint
    animation = windowsIn,     1,     4.1,   easeOutQuint, popin 87%
    animation = windowsOut,    1,     1.49,  linear,       popin 87%
    animation = fadeIn,        1,     1.73,  almostLinear
    animation = fadeOut,       1,     1.46,  almostLinear
    animation = fade,          1,     3.03,  quick
    animation = layers,        1,     3.81,  easeOutQuint
    animation = layersIn,      1,     4,     easeOutQuint, fade
    animation = layersOut,     1,     1.5,   linear,       fade
    animation = fadeLayersIn,  1,     1.79,  almostLinear
    animation = fadeLayersOut, 1,     1.39,  almostLinear
    animation = workspaces,    1,     1.94,  almostLinear, fade
    animation = workspacesIn,  1,     1.21,  almostLinear, fade
    animation = workspacesOut, 1,     1.94,  almostLinear, fade
    animation = zoomFactor,    1,     7,     quick
}

# Ref https://wiki.hypr.land/Configuring/Workspace-Rules/
# "Smart gaps" / "No gaps when only"
# uncomment all if you wish to use that.
# workspace = w[tv1], gapsout:0, gapsin:0
# workspace = f[1], gapsout:0, gapsin:0
```

```
# windowrule {
#     name = no-gaps-wtv1
#     match:float = false
#     match:workspace = w[tv1]
#
#     border_size = 0
#     rounding = 0
# }
#
# windowrule {
#     name = no-gaps-f1
#     match:float = false
#     match:workspace = f[1]
#
#     border_size = 0
#     rounding = 0
# }

# See https://wiki.hypr.land/Configuring/Dwindle-Layout/ for more
dwindle {
    preserve_split = true # You probably want this
}

# See https://wiki.hypr.land/Configuring/Master-Layout/ for more
master {
    new_status = master
}

# https://wiki.hypr.land/Configuring/Variables/#misc
misc {
    force_default_wallpaper = -1 # Set to 0 or 1 to disable the anime mascot wallpapers
    disable_hyprland_logo = false # If true disables the random hyprland logo / anime g
}


#############
### INPUT ###
#############

# https://wiki.hypr.land/Configuring/Variables/#input
```

```
input {
    kb_layout = us
    kb_variant = intl
    kb_model =
    kb_options =
    kb_rules =

    follow_mouse = 1

    sensitivity = 0.5 # -1.0 - 1.0, 0 means no modification.

    touchpad {
        natural_scroll = false
    }
}

# See https://wiki.hypr.land/Configuring/Gestures
gesture = 3, horizontal, workspace

# Example per-device config
# See https://wiki.hypr.land/Configuring/Keywords/#per-device-input-configs for more
device {
    name = epic-mouse-v1
    sensitivity = -0.5
}


####################
### KEYBINDINGS ###
####################


# Screenshots
bind = , Print, exec, hyprshot -m region
bind = SHIFT, Print, exec, hyprshot -m output -m active -o ~/Pictures

# See https://wiki.hypr.land/Configuring/Keywords/
$mainMod = SUPER # Sets "Windows" key as main modifier

# yprlock
```

```
bind = $mainMod, Escape, exec, hyprlock

# Example binds, see https://wiki.hypr.land/Configuring/Binds/ for more
bind = $mainMod, Return, exec, $terminal
bind = $mainMod SHIFT, Q, killactive,
bind = $mainMod, M, exec, command -v hyprshutdown >/dev/null 2>&1 && hyprshutdown || hy
bind = $mainMod SHIFT, space, togglefloating,
bind = $mainMod, D, exec, $menu
bind = $mainMod, P, pseudo, # dwindle

# Emacs
bind = $mainMod, E, exec, emacs

# Move focus with mainMod + arrow keys
bind = $mainMod, H, hy3:movefocus, l
bind = $mainMod, L, hy3:movefocus, r
bind = $mainMod, K, hy3:movefocus, u
bind = $mainMod, J, hy3:movefocus, d

# Make group (Stacking HY3) we acuerdate de esto del plugin
bind = $mainMod, S, hy3:makegroup, tab

# Move current window left, right, up and down
bind = $mainMod SHIFT, H, hy3:movewindow, l
bind = $mainMod SHIFT, L, hy3:movewindow, r
bind = $mainMod SHIFT, K, hy3:movewindow, u
bind = $mainMod SHIFT, J, hy3:movewindow, d

# Switch workspaces with mainMod + [0-9]
bind = $mainMod, 1, workspace, 1
bind = $mainMod, 2, workspace, 2
bind = $mainMod, 3, workspace, 3
bind = $mainMod, 4, workspace, 4
bind = $mainMod, 5, workspace, 5
bind = $mainMod, 6, workspace, 6
bind = $mainMod, 7, workspace, 7
bind = $mainMod, 8, workspace, 8
bind = $mainMod, 9, workspace, 9
bind = $mainMod, 0, workspace, 10
```

```
# Move active window to a workspace with mainMod + SHIFT + [0-9]
bind = $mainMod SHIFT, 1, movetoworkspace, 1
bind = $mainMod SHIFT, 2, movetoworkspace, 2
bind = $mainMod SHIFT, 3, movetoworkspace, 3
bind = $mainMod SHIFT, 4, movetoworkspace, 4
bind = $mainMod SHIFT, 5, movetoworkspace, 5
bind = $mainMod SHIFT, 6, movetoworkspace, 6
bind = $mainMod SHIFT, 7, movetoworkspace, 7
bind = $mainMod SHIFT, 8, movetoworkspace, 8
bind = $mainMod SHIFT, 9, movetoworkspace, 9
bind = $mainMod SHIFT, 0, movetoworkspace, 10

# Scroll through existing workspaces with mainMod + scroll
bind = $mainMod, mouse_down, workspace, e+1
bind = $mainMod, mouse_up, workspace, e-1

# Move/resize windows with mainMod + LMB/RMB and dragging
bindm = $mainMod, mouse:272, hy3:movewindow
bindm = $mainMod, mouse:273, resizewindow

# Laptop multimedia keys for volume and LCD brightness
bindel = ,XF86AudioRaiseVolume, exec, wpctl set-volume -l 1 @DEFAULT_AUDIO_SINK@ 5%+
bindel = ,XF86AudioLowerVolume, exec, wpctl set-volume @DEFAULT_AUDIO_SINK@ 5%-
bindel = ,XF86AudioMute, exec, wpctl set-mute @DEFAULT_AUDIO_SINK@ toggle
bindel = ,XF86AudioMicMute, exec, wpctl set-mute @DEFAULT_AUDIO_SOURCE@ toggle
bindel = ,XF86MonBrightnessUp, exec, brightnessctl -e4 -n2 set 5%+
bindel = ,XF86MonBrightnessDown, exec, brightnessctl -e4 -n2 set 5%-

# Requires playerctl
bindl = , XF86AudioNext, exec, playerctl next
bindl = , XF86AudioPause, exec, playerctl play-pause
bindl = , XF86AudioPlay, exec, playerctl play-pause
bindl = , XF86AudioPrev, exec, playerctl previous

##############################
### WINDOWS AND WORKSPACES ###
##############################

# See https://wiki.hypr.land/Configuring/Window-Rules/ for more
# See https://wiki.hypr.land/Configuring/Workspace-Rules/ for workspace rules
```

```
# Example windowrules that are useful

windowrule {
    # Ignore maximize requests from all apps. You'll probably like this.
    name = suppress-maximize-events
    match:class = .*

    suppress_event = maximize
}

windowrule {
    # Fix some dragging issues with XWayland
    name = fix-xwayland-drags
    match:class = ^$
    match:title = ^$
    match:xwayland = true
    match:float = true
    match:fullscreen = false
    match:pin = false

    no_focus = true
}

# Hyprland-run windowrule
windowrule {
    name = move-hyprland-run

    match:class = hyprland-run

    move = 20 monitor_h-120
    float = yes
}


    '';
```

6.4 Configuraciones

Las configuraciones principales de este gestor de ventanas se concentran en un sólo archivo que vive en ~/.config/hypr/hyprland.conf. De todas formas, existe otra variedad de archivos para cada uno de los módulos que vamos a ir configurando para crear mi configuración final. Por lo que iré avanzando feature por feature -aclarando el archivo que estamos modificando- hasta terminar.

6.4.1 Configuración de los monitores

Tengo que reconocer que configurar los monitores en Hyprland es algo que me sorprendió de lo sencillo y tan bien logrado que está. En otros gestores de ventanas, generalmente tendríamos que recurrir a scripts externos de xrandr que realicen automáticamente estas configuraciiones -y meternos con xrandr es algo que todavía me despierta sudando en noches frías-. Por lo que el enterarme de que en Hyprland, esto es algo que vamos a configurar desde su archivo de configuración, fue algo bastante agradable.

Yo en mi laptop solamente tengo el potencial de dos pantallas: una en HDMI y la misma interna de la laptop. Entrando a la sección de documentación de Hyprland que lo explica vemos que tenemos varios parámetros para configurar de un monitor. Partiendo de la generalidad de que todo se ve como:

```
monitor = name, resolution, position, scale
```

Quiero que el posible monitor en HDMI quede "arriba" de mi monitor principal y use la resolución preferida por el monitor, para que se vea lo mejor posible. Esto porque generalmente este slot lo suelo usar para dos cosas:

- Para mi monitor principal en casa, el cual está posicionado para que pueda trabajar de pie con él.
- Para conectarme a proyectores de la escuela a exponer y dar clases. Generalmente todos proyectan hacia la mitad superior de una pared... cuando estoy sentado esto es arriba de mi nivel de vista.

También cambio la escala utilizada por el monitor a algun número alrededor de 0.9 para que los elementos de la pantalla se encojan un poco y quepa más información. Y finalmente, mantengo comentada una sección para poder hacer que el monitor HDMI sea un espejo del monitor de mi laptop.

Para mi monitor nativo y para otros que puedan venir, sólo me interesa la alta resolución.

Quedando de esta manera:

```
monitor= HDMI-A-1,preferred,auto-up,0.9 #,mirror, eDP-1
monitor=,highres,auto,auto
```

6.4.2 Defaults

En teoría en esta sección debería de configurar varios defaults. Pero tengo la sospecha de que me perdí de esto por no tener un sistema limpio. Por ahora tengo esto, pero probablemente cuando regrese con una configuración limpia tendré que revisar esta sección.

```
$terminal = alacritty
$menu = ~/.config/rofi/launchers/type-1/launcher.sh
```

6.4.3 Wallpaper

Aquí empezamos con aquellas aplicaciones que deberían de arrancar al iniciar el entorno. Para esto, usamos el comando exec-once.

Comenzamos con el fondo de pantalla, que como de costumbre en gestores de ventanas, es un programa que ejecutamos al inicio.

```
exec-once = hyprpaper
```

Y ya hablando de fondos de pantalla, podemos asomarnos un poco al archivo de configuración correspondiente para hyprpaper. Este vive en ~/.config/hypr/hyprpaper.conf. Y es que de hecho es un archivo bastante simple que se lee bastante sencillo

```
wallpaper {
    monitor =
    path = ~/PersonalDotFiles/Arch/wallpapers/wallpaperLaptop.jpg
    fit_mode = fill
}
```

La herramienta tiene mucho potencial para crear cosas bastante fancys como wallpapers que se van rotando, sets de estos para cada uno de los monitores que tengas y ajá... Yo por mi parte soy una persona bastante simple y solamente quiero que mi fondo de pantalla de siempre se vea en todos mis monitores.

6.4.4 Waybar

Esta es la barra de estado por default de Hyprland y la que utilizo porque en realidad no necesito mucho más. Siendo honestos, mantengo la configuración por default excepto por alguna configuración extremadamente puntual que ni siquiera podría recordar que hice, así que voy a romper un poco el paradigma letrado de este y pegar directamente la configuración. Si algún día realizo algún cambio a los defaults que merezca ser señalado, modificaré esta sección.

```
// -*- mode: jsonc -*-
{
    // "layer": "top", // Waybar at top layer
    // "position": "bottom", // Waybar position (top|bottom|left|right)
    "height": 20, // Waybar height (to be removed for auto height)
    // "width": 1280, // Waybar width
    "spacing": 4, // Gaps between modules (4px)
    // Choose the order of the modules
    "modules-left": [
        "hyprland/workspaces",
        "hyprland/submap",
        "sway/scratchpad",
        "custom/media"
    ],
    "modules-center": [
        "sway/window"
    ],
    "modules-right": [
        "mpd",
        "idle_inhibitor",
        "pulseaudio",
        "network",
        "power-profiles-daemon",
        "cpu",
        "memory",
        "temperature",
        "backlight",
        "keyboard-state",
        "sway/language",
        "battery",
        "battery#bat2",
        "clock",
```

```
    "tray",
    "custom/power"
],
// Modules configuration
// "hyprland/workspaces": {
//     "disable-scroll": true,
//     "all-outputs": true,
//     "warp-on-scroll": false,
//     "format": "{name}: {icon}",
//     "format-icons": {
//         "1": "",
//         "2": "",
//         "3": "",
//         "4": "",
//         "5": "",
//         "urgent": "",
//         "focused": "",
//         "default": ""
//     }
// },
"keyboard-state": {
    "numlock": true,
    "capslock": true,
    "format": "{name} {icon}",
    "format-icons": {
        "locked": "",
        "unlocked": ""
    }
},
"hyprland/submap": {
    "format": "<span style=\"italic\">{}</span>"
},
"sway/scratchpad": {
    "format": "{icon} {count}",
    "show-empty": false,
    "format-icons": ["", ""],
    "tooltip": true,
    "tooltip-format": "{app}: {title}"
},
"mpd": {
```

```
    "format": "{stateIcon} {consumeIcon}{randomIcon}{repeatIcon}{singleIcon}{artist
    "format-disconnected": "Disconnected ",
    "format-stopped": "{consumeIcon}{randomIcon}{repeatIcon}{singleIcon}Stopped ",
    "unknown-tag": "N/A",
    "interval": 5,
    "consume-icons": {
        "on": " "
    },
    "random-icons": {
        "off": "<span color=\"#f53c3c\"></span> ",
        "on": " "
    },
    "repeat-icons": {
        "on": " "
    },
    "single-icons": {
        "on": "1 "
    },
    "state-icons": {
        "paused": "",
        "playing": ""
    },
    "tooltip-format": "MPD (connected)",
    "tooltip-format-disconnected": "MPD (disconnected)"
},
"idle_inhibitor": {
    "format": "{icon}",
    "format-icons": {
        "activated": "",
        "deactivated": ""
    }
},
"tray": {
    // "icon-size": 21,
    "spacing": 10,
    // "icons": {
    //   "blueman": "bluetooth",
    //   "TelegramDesktop": "$HOME/.local/share/icons/hicolor/16x16/apps/telegram.p
    // }
},
```

```
"clock": {
    // "timezone": "America/New_York",
    "tooltip-format": "<big>{:%Y %B}</big>\n<tt><small>{calendar}</small></tt>",
    "format-alt": "{:%Y-%m-%d}"
},
"cpu": {
    "format": "{usage}% ",
    "tooltip": false
},
"memory": {
    "format": "{}% "
},
"temperature": {
    // "thermal-zone": 2,
    // "hwmon-path": "/sys/class/hwmon/hwmon2/temp1_input",
    "critical-threshold": 80,
    // "format-critical": "{temperatureC}°C {icon}",
    "format": "{temperatureC}°C {icon}",
    "format-icons": ["", "", ""]
},
"backlight": {
    // "device": "acpi_video1",
    "format": "{percent}% {icon}",
    "format-icons": ["", "", "", "", "", "", "", "", ""]
},
"battery": {
    "states": {
        // "good": 95,
        "warning": 30,
        "critical": 15
    },
    "format": "{capacity}% {icon}",
    "format-full": "{capacity}% {icon}",
    "format-charging": "{capacity}% ",
    "format-plugged": "{capacity}% ",
    "format-alt": "{time} {icon}",
    // "format-good": "", // An empty format will hide the module
    // "format-full": "",
    "format-icons": ["", "", "", "", ""]
},
```

```
"battery#bat2": {
    "bat": "BAT2"
},
"power-profiles-daemon": {
  "format": "{icon}",
  "tooltip-format": "Power profile: {profile}\nDriver: {driver}",
  "tooltip": true,
  "format-icons": {
    "default": "",
    "performance": "",
    "balanced": "",
    "power-saver": ""
  }
},
"network": {
    // "interface": "wlp2*", // (Optional) To force the use of this interface
    "format-wifi": "{essid} ({signalStrength}%) ",
    "format-ethernet": "{ipaddr}/{cidr} ",
    "tooltip-format": "{ifname} via {gwaddr} ",
    "format-linked": "{ifname} (No IP) ",
    "format-disconnected": "Disconnected ",
    "format-alt": "{ifname}: {ipaddr}/{cidr}"
},
"pulseaudio": {
    // "scroll-step": 1, // %, can be a float
    "format": "{volume}% {icon} {format_source}",
    "format-bluetooth": "{volume}% {icon} {format_source}",
    "format-bluetooth-muted": " {icon} {format_source}",
    "format-muted": " {format_source}",
    "format-source": "{volume}% ",
    "format-source-muted": "",
    "format-icons": {
        "headphone": "",
        "hands-free": "",
        "headset": "",
        "phone": "",
        "portable": "",
        "car": "",
        "default": ["", "", ""]
    },
```

```
        "on-click": "pavucontrol"
    },
    "custom/media": {
        "format": "{icon} {text}",
        "return-type": "json",
        "max-length": 40,
        "format-icons": {
            "spotify": "",
            "default": ""
        },
        "escape": true,
        "exec": "$HOME/.config/waybar/mediaplayer.py 2> /dev/null" // Script in resourc
        // "exec": "$HOME/.config/waybar/mediaplayer.py --player spotify 2> /dev/null"
    },
    "custom/power": {
        "format" : " ",
  "tooltip": false,
  "menu": "on-click",
  "menu-file": "$HOME/.config/waybar/power_menu.xml", // Menu file in resources folder
  "menu-actions": {
   "shutdown": "shutdown",
   "reboot": "reboot",
   "suspend": "systemctl suspend",
   "hibernate": "systemctl hibernate"
  }
    }
}
```

Esta herramienta además de la configuración requiere también una hoja de estilos, la cual igual dejaré por el momento por default.

```
\* {
    /* ‘otf-font-awesome‘ is required to be installed for icons */
    font-family: FontAwesome, Roboto, Helvetica, Arial, sans-serif;
    font-size: 13px;
}

window#waybar {
    background-color: rgba(43, 48, 59, 0.5);
    border-bottom: 3px solid rgba(100, 114, 125, 0.5);
```

```
    color: #ffffff;
    transition-property: background-color;
    transition-duration: .5s;
}

window#waybar.hidden {
    opacity: 0.2;
}

/*
window#waybar.empty {
    background-color: transparent;
}
window#waybar.solo {
    background-color: #FFFFFF;
}
*/

window#waybar.termite {
    background-color: #3F3F3F;
}

window#waybar.chromium {
    background-color: #000000;
    border: none;
}

button {
    /* Use box-shadow instead of border so the text isn't offset */
    box-shadow: inset 0 -3px transparent;
    /* Avoid rounded borders under each button name */
    border: none;
    border-radius: 0;
}

/* https://github.com/Alexays/Waybar/wiki/FAQ#the-workspace-buttons-have-a-strange-hove
button:hover {
    background: inherit;
    box-shadow: inset 0 -3px #ffffff;
}
```

```
/* you can set a style on hover for any module like this */
#pulseaudio:hover {
    background-color: #a37800;
}

#workspaces button {
    padding: 0 5px;
    background-color: transparent;
    color: #ffffff;
}

#workspaces button:hover {
    background: rgba(0, 0, 0, 0.2);
}

#workspaces button.focused {
    background-color: #64727D;
    box-shadow: inset 0 -3px #ffffff;
}

#workspaces button.urgent {
    background-color: #eb4d4b;
}

#mode {
    background-color: #64727D;
    box-shadow: inset 0 -3px #ffffff;
}

#clock,
#battery,
#cpu,
#memory,
#disk,
#temperature,
#backlight,
#network,
#pulseaudio,
#wireplumber,
```

```
#custom-media,
#tray,
#mode,
#idle_inhibitor,
#scratchpad,
#power-profiles-daemon,
#mpd {
    padding: 0 10px;
    color: #ffffff;
}

#window,
#workspaces {
    margin: 0 4px;
}

/* If workspaces is the leftmost module, omit left margin */
.modules-left > widget:first-child > #workspaces {
    margin-left: 0;
}

/* If workspaces is the rightmost module, omit right margin */
.modules-right > widget:last-child > #workspaces {
    margin-right: 0;
}

#clock {
    background-color: #64727D;
}

#battery {
    background-color: #ffffff;
    color: #000000;
}

#battery.charging, #battery.plugged {
    color: #ffffff;
    background-color: #26A65B;
}
```

```
@keyframes blink {
    to {
        background-color: #ffffff;
        color: #000000;
    }
}

/* Using steps() instead of linear as a timing function to limit cpu usage */
#battery.critical:not(.charging) {
    background-color: #f53c3c;
    color: #ffffff;
    animation-name: blink;
    animation-duration: 0.5s;
    animation-timing-function: steps(12);
    animation-iteration-count: infinite;
    animation-direction: alternate;
}

#power-profiles-daemon {
    padding-right: 15px;
}

#power-profiles-daemon.performance {
    background-color: #f53c3c;
    color: #ffffff;
}

#power-profiles-daemon.balanced {
    background-color: #2980b9;
    color: #ffffff;
}

#power-profiles-daemon.power-saver {
    background-color: #2ecc71;
    color: #000000;
}

label:focus {
    background-color: #000000;
}
```

```
#cpu {
    background-color: #2ecc71;
    color: #000000;
}

#memory {
    background-color: #9b59b6;
}

#disk {
    background-color: #964B00;
}

#backlight {
    background-color: #90b1b1;
}

#network {
    background-color: #2980b9;
}

#network.disconnected {
    background-color: #f53c3c;
}

#pulseaudio {
    background-color: #f1c40f;
    color: #000000;
}

#pulseaudio.muted {
    background-color: #90b1b1;
    color: #2a5c45;
}

#wireplumber {
    background-color: #fff0f5;
    color: #000000;
}
```

```
#wireplumber.muted {
    background-color: #f53c3c;
}

#custom-media {
    background-color: #66cc99;
    color: #2a5c45;
    min-width: 100px;
}

#custom-media.custom-spotify {
    background-color: #66cc99;
}

#custom-media.custom-vlc {
    background-color: #ffa000;
}

#temperature {
    background-color: #f0932b;
}

#temperature.critical {
    background-color: #eb4d4b;
}

#tray {
    background-color: #2980b9;
}

#tray > .passive {
    -gtk-icon-effect: dim;
}

#tray > .needs-attention {
    -gtk-icon-effect: highlight;
    background-color: #eb4d4b;
}
```

```
#idle_inhibitor {
    background-color: #2d3436;
}

#idle_inhibitor.activated {
    background-color: #ecf0f1;
    color: #2d3436;
}

#mpd {
    background-color: #66cc99;
    color: #2a5c45;
}

#mpd.disconnected {
    background-color: #f53c3c;
}

#mpd.stopped {
    background-color: #90b1b1;
}

#mpd.paused {
    background-color: #51a37a;
}

#language {
    background: #00b093;
    color: #740864;
    padding: 0 5px;
    margin: 0 5px;
    min-width: 16px;
}

#keyboard-state {
    background: #97e1ad;
    color: #000000;
    padding: 0 0px;
    margin: 0 5px;
    min-width: 16px;
```

```
}

#keyboard-state > label {
    padding: 0 5px;
}

#keyboard-state > label.locked {
    background: rgba(0, 0, 0, 0.2);
}

#scratchpad {
    background: rgba(0, 0, 0, 0.2);
}

#scratchpad.empty {
 background-color: transparent;
}

#privacy {
    padding: 0;
}

#privacy-item {
    padding: 0 5px;
    color: white;
}

#privacy-item.screenshare {
    background-color: #cf5700;
}

#privacy-item.audio-in {
    background-color: #1ca000;
}

#privacy-item.audio-out {
    background-color: #0069d4;
}
```

Y finalmente agregamos su entrada de exec-once al archivo de configuración principal de Hyprland:

```
exec-once = waybar
```

6.4.5 Variables de entorno

Necesitamos gestionar la creación de algunas variables de entorno que serán necesarias para que los programas bajo esta configuración puedan funcionar correctamente. Para hacer esto, Hyprland nos proporciona su utilidad env la cual crea las variables de entorno directamente en el proceso de Hyprland, y todos los procesos hijos las heredarán sin problemas.

Pero también hay que considerar la sutileza de que necesitamos también crear variables de entorno en procesos que no serán hijos del proceso Hyprland, tales como los creados por Systemd y el D-Bus. En este caso, necesitamos también una utilidad que deberá pasarle estas variables de entorno (un shot cada que diga variables de entorno) a ambos procesos para que sus hijos también las tengan. Esto se logra con:

```
exec-once=dbus-update-activation-environment --systemd WAYLAND_DISPLAY XDG_CURRENT_DESK
```

Esto lo que hace es activar dicha utilidad (dbus-update-activation-environment) y decirle que tome las variables $\text{WAYLAND}_{\text{DISPLAY}}$ y $\text{XDG}_{\text{CURRENTDESKTOP}}$ y se las pase tanto a D-Bus como a Systemd para que los procesos hijos de estos las tengan también.

Ya de paso no sobra explicar qué hace cada una de estas dos variables:

- La variable $\text{WAYLAND}_{\text{DISPLAY}}$ es la que le define a las aplicaciones que estamos usando Wayland y no X11, y que el socket al que deben de enviar los eventos es el dado. Por ejemplo, suele tomar el valor de wayland-1 o wayland-0 porque no suelo tener otra sesión de Wayland en mi computadora.

- La variable $\text{XDG}_{\text{CURRENTDESKTOP}}$ dicta el tipo de escritorio que se está utilizando en estos momentos. De aquí se agarran los procesos para determinar tanto el portal como ciertas preferencias y estéticas.

Ambas son creadas por Hyprland al iniciar, así que esta linea en realidad lo que hace no es declararlas si no pasárselas a Systemd y D-Bus.

Las que si vamos a crear son:

```
env = XCURSOR_SIZE,24
env = HYPRCURSOR_SIZE,24
```

Porque con estas vamos a determinar el tamaño del cursor. Usamos dos porque $\text{XCURSOR}_{\text{SIZE}}$ es la utilizada por aplicaciones que tengan que correr bajo Xwayland (la capa de compatibilidad de Wayland con X11) para funcionar.

6.4.6

7 Dropbox

Utilizo Dropbox muy seguido para mantener la sincronización de mi agendaORG

7.1 Instalación

```
maestral
```

8 Virtual Box y QEMU/KVM

Debido a varias aplicaciones de la escuela, necesitaré constantemente recurrir a máquinas virtuales de diversos índoles y distribuciones. Tanto por temas de materias como por temas de trabajo y cosas extracurriculares.

Particularmente sobre QEMU/KVM debo de decir que esta es mi opción predilecta. Ya que es la plataforma de virtualización oficial de Linux, tiene muchas optimizaciones a nivel kernel y sí puedo notar que me ofrece un rendimiento bastante bueno.

```
virtualisation.libvirtd.enable = true;
programs.virt-manager.enable = true;
services.spice-vdagentd.enable = true;
```

Por otro lado, también necesito Virtual Box. Sobre todo para temas de redes, ya que la configuración de máquinas virtuales en modo bridge y modo NAT es bastante sencilla y rápida en este aplicativo, cosa no tan cierta para la opción anterior. Es decir, para temas donde necesito configurar redes, utilizo Virtual Box, pero para el otro 99% de los casos, prefiero QEMU/KVM.

```
virtualisation.virtualbox.host.enable = true;
virtualisation.virtualbox.host.enableExtensionPack = true;
```

9 Zen Browser

9.1 Instalación

Ya que no hay paquete oficial, nos habremos de servir de un Flake.

```
zen-browser = {
    url = "github:youwen5/zen-browser-flake/458cd16116c362a5e92d5db6bdc3599e4e8b1f4a";
    inputs.nixpkgs.follows = "nixpkgs";
  };
```

Y luego lo declaramos como paquete de sistema

```
inputs.zen-browser.packages.${pkgs.stdenv.hostPlatform.system}.default
```

10 Programación con IA

Para el tema de programación con IA me voy a servir de tres herramientas: Gptel, OpenCode, y copilot.el. Las primeras dos van a utilizar modelos provenientes de Groq, que es mi enrutador de modelos de lenguaje predilecto. Mientras que la última es una implementación de Github Copilot para Emacs, así que en realidad no tengo mucha elección sobre el modelo de lenguaje a utilizar.

Utilizo estas tres porque me gusta que las features de IA formen parte de mi flujo de trabajo pero no tomen un rol esencial dentro del mismo. Quiero seguir usando Emacs, que es mi editor de texto favorito justamente porque es bastante reducido y sólo contiene lo que necesito, pero a la vez quiero tener en fácil alcance las features de IA para mantenerme competitivo en el mundo laboral actual; el cual evidentemente cada vez requerirá un uso más predominante de la IA en nuestros flujos de trabajo. Este flujo de trabajo es la forma que tengo de hacerlo "bajo mis términos". Aunque, dicho sea de paso, OpenCode es un agente autónomo que toma con bastante fuerza el control del proyecto en el que lo use. Particularmente con ciertas herramientas que veré más adelante que le agrego; por esto, lo uso sólo para ciertos casos de experimentación o sprints extremadamente rápidos.

10.1 Gptel

> gptel is a simple Large Language Model chat client for Emacs, with support for many models and backends. It works in the spirit of Emacs, available at any time and uniformly in any buffer.

Este es un plugin de emacs que voy a utilizar para interactuar con IA dentro del mismo editor de texto. Para esto, solamente cargo las siguientes lineas dentro de mi config.el:

```
(use-package! gptel
  :config
  (setq gptel-backend
        (gptel-make-openai "Groq"
          :host "api.groq.com"
          :endpoint "/openai/v1/chat/completions"
          :stream t
          :key (lambda ()
       (with-temp-buffer
          (insert-file-contents "~/.groq-secret")
          (if (string-match "export GROQ_API_KEY=\"\\(.*\\)\"" (buffer-string))
              (match-string 1 (buffer-string))
            (string-trim (buffer-string)))))
          :models '(llama-3.3-70b-versatile
                    llama-3.1-8b-instant
                    mixtral-8x7b-32768)))

  ;; 2. Establecer el modelo por defecto
  (setq gptel-model 'llama-3.3-70b-versatile))
```

Estas lineas son la configuración básica para que el plugin funcione correctamente. Hacen poco más que declarar el backend (uso Groq) y el modelo de lenguaje por default que usaré de este mismo.

Luego, también dentro de config.el, agrego algunas opciones de configuración más relacionadas directamente con el funcionamiento de la herramienta.

La primera es ajustar la variable `gptel-context-restrict-to-project-files` a `nil`. Esto ya que, en ciertos casos necesito ignorar ciertas restricciones que la herramienta coloca para agregar al contexto del modelo ciertos archivos que sean parte de un repositorio en git.

```
(setq gptel-context-restrict-to-project-files 'nil)
```

Y cargamos el paquete declarando lo siguiente en packages.el

```
(package! gptel)
```

Evidentemente voy a omitir la API Key en esta especificación.

Finalmente, definimos los bindings de teclado:

```
(map! :leader
      :prefix "l" ; "l" de LLM
      :desc "Gptel Menu" "l" #'gptel-menu
      :desc "Gptel Send" "s" #'gptel-send
      :desc "Gptel Chat" "c" #'gptel)
```

10.2 OpenCode

> OpenCode is an open source agent that helps you write code in your terminal, IDE, or desktop.

Esta es mi integración para linea de comandos y mi agente autónomo para proyectos que busco realizar puramente con IA, o que busco que salgan en sprints extremadamente rápidos. Es mi alternativa Open Source a tener Claude Code, con miras a en un futuro autohostear los modelos de lenguaje que utilice.

Para comenzar, la instalamos como paquete de usuario en nuestro sistema:

```
opencode
```

El resto de la configuración inicial es desde la interfaz (TUI) de la herramienta. Consiste en primicia en especificarle qué proveedor de IA utilizar (en mi caso Groq), especificarle la API key para este uso, y seleccionar el modelo. El asistente de instalación se inicia con `opencode` y luego con `/connect`

10.3 Copilot.el

Esta es una implementación de Github Copilot dentro de Emacs. No hace más que proporcionar los autocompletados pasivos que Copilot da, que de hecho es lo suficiente para recibir un boost bastante bueno de productividad sin ceder autonomía.

Empezamos declarando el uso del paquete dentro de `packages.el`

```
(package! copilot
  :recipe (:host github :repo "copilot-emacs/copilot.el" :files ("*.el")))
```

Y luego, en la configuración, vamos a definir la aceptación de los autocompletados de Copilot mediante TAB y sus distintos modos de hacerlo.

```
(use-package! copilot
  :hook (prog-mode . copilot-mode)
  :bind (:map copilot-completion-map
              ("<tab>" . 'copilot-accept-completion)
              ("TAB" . 'copilot-accept-completion)
              ("C-TAB" . 'copilot-accept-completion-by-word)
              ("C-<tab>" . 'copilot-accept-completion-by-word)))
```

> Then run M-x copilot-install-server and M-x copilot-login. That's it!

11 Lenguajes de programación

Para cada lenguaje de programación necesito configurar mi entorno en base a las herramientas necesarias vinculadas al flujo de trabajo de dicho lenguaje. En esta sección, veremos uno a uno los requerimentos que necesito para empezar a trabajar con todos los lenguajes de programación con los que trabaje en ese momento.

11.1 C y C++

La madre de todos los lenguajes modernos y su hijo cuestionable. Lo uso sobre todo para proyectos de bajo nivel, como por ejemplo Arduino y cosas IoT.

Empezamos por declarar su compilador como paquete de sistema.

```
gcc
```

Y agregamos soporte a mi IDE para estos lenguajes, con su respectivo módulo lang.

```
(cc +lsp)
```

11.2 Golang

Este es el lenguaje de programación con el que he estado más fijado últimamente. Me gusta mucho su sintaxis y me agrada bastante el estilo de escritura y los comentarios de sus mantenedores en su Dev blog

Lo primero que hago es declarar su compilador dentro de mis **userPackages**

```
go
```

No fijo la versión de Golang a ninguna en particular, pues, por lo general, siempre estaré trabajando en la última versión para mis desarrollos personales. Si en algún caso necesitara una versión particular para algún proyecto, creo que mi opción será usar **nix-shells** o entornos virtuales para ese caso particular.

Luego, para preparar mi IDE (Emacs) para trabajar con Go, agrego su módulo respectivo a los módulos de lenguajes.

```
(go +lsp)
```

11.3 Javascript

Casi no suelo trabajar con Javascript. Tengo poca experiencia con este lenguaje porque mi opinión a título personal es que nunca debió salir del frontend, pero ajá.

Necesito Node para ciertas dependencias de mi sistema, así que lo declaro como systemPackage.

```
  nodejs
```

Luego, para preparar mi IDE (Emacs) para trabajar con este lenguaje, agrego su módulo de lenguaje a mi configuración.

```
javascript
```

11.4 Python

En mis primeros semestres de universidad llegué a caer bajo las ideas de que "Python no es un lenguaje de programación real" y pasé a evitarlo como la peste durante un rato. Todo cambió cuando, gracias a Dios, mi lóbulo frontal se terminó de desarrollar y me dí cuenta de que no tiene sentido que existan guerras de lenguajes siempre que nunca endiosemos ni odiemos conscientemente ningún lenguaje de programación y los utilicemos como lo que son: herramientas para realizar un trabajo, con sus propios casos de uso y sus ventajas y desventajas.

Y el caso de uso que le suelo a dar Python es para scripts rápidos, cuentas rápidas en su intérprete (que he pasado a usar como calculadora) y para obviamente correr ciertas herramientas desarrolladas en el mismo.

Hablando de su intérprete, lo agregamos a los paquetes de sistema.

```
python3
```

Luego agregamos soporte para este lenguaje en mi IDE (Emacs), agregando su módulo a los módulos de lenguaje en configuraciones.

```
python
```

11.4.1 Pyenv

Pyenv es una herramienta que he usado durante algo de tiempo -probablemente más de un año- porque me permite cambiar entre versiones de Python de una manera bastante rápida y conveniente. Es, como su página lo dice, "simple, unobtrusive, and follows the UNIX tradition of single-purpose tools that do one thing well.".

Lo declaro como paquete de usuario en la configuración de NixOS

```
pyenv
```

Luego agregro a mi `.zshrc` (véase OhMyZSH) algunas líneas que vienen también en su página. Las cuales son las siguientes:

```
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

11.4.2 Conda

Anaconda es una distribución de Python y R orientada a ciencia de datos y machine learning. En un principio la usaba para montarme entornos virtuales de Python.

Esta es una suerte de sección legacy porque en realidad desde que uso Virtual Envs y Pyenv he dejado de usar Anaconda, pero creo que no sobra tampoco. Agregamos las siguientes líneas a mi `.zshrc`

```
# >>> conda initialize >>>
# !! Contents within this block are managed by 'conda init' !!
```

```
__conda_setup="$('/home/mrtaichi/anaconda3/bin/conda' 'shell.zsh' 'hook' 2> /dev/null)"
if [ $? -eq 0 ]; then
    eval "$__conda_setup"
else
    if [ -f "/home/mrtaichi/anaconda3/etc/profile.d/conda.sh" ]; then
        . "/home/mrtaichi/anaconda3/etc/profile.d/conda.sh"
    else
        export PATH="/home/mrtaichi/anaconda3/bin:$PATH"
    fi
fi
unset __conda_setup
# <<< conda initialize <<<
```

Y luego declaro el paquete correspondiente como paquete de usuario.

```
conda
```

11.5 Haskell

Haskell es un lenguaje de programación que, no lo puedo negar, al inicio me parecía bastante pesado y tedioso. Cuando uno proviene de los lenguajes imperativos, es complicado comenzar a pensar de manera funcional.

Pero conforme más lo uso y más me sumerjo en las matemáticas y el cariño detrás de su compilador y en general de todo este paradigma de programación, más me gusta usarlo. Tal vez algún día le invierta más tiempo a su uso, quizás en forma de algún proyecto interesante.

Cuando usaba Arch, era tan sencillo como tener GHCup instalado, pero al investigar 5 minutos sobre cómo empezar a trabajar con Haskell en NixOS, me doy cuenta de que es un problema que voy a procrastinar hasta que me toque desarrollar un proyecto en este lenguaje. Por lo que esta sección queda pendiente.

11.6 Clojure

Otro lenguaje funcional, pero esta vez hecho para funcionar en la máquina virtual de Java. Necesito trabajar más en él, ya que no tengo la memoria de por qué tengo un entorno de trabajo preparado para el mismo.

Comenzamos con declarar como userPackage las dos cosas esenciales de este lenguaje: Su compilador y su herramienta de gestión de proyectos principal.

```
clojure
leiningen
```

Y, como siempre, agrego su módulo de lenguaje a mi IDE (Emacs)

```
clojure
```

11.7 Java

Qué sería un estudiante de CC sin Java?

Agrego su último compilador a la fecha (OpenJDK25) a mis userPackages, junto con Maven, mi gestor de proyectos favorito.

```
pkgs.javaPackages.compiler.openjdk25
maven
```

Y adapto mi IDE, agregando su respectivo módulo de lenguaje.

```
(java +lsp)
```

12 Ciberseguridad

Otro de los grandes pilares de mi especialización dentro de mi carrera, y de hecho el que más frutos me ha rendido hasta el momento en el que redacto esta sección es la ciberseguridad. Para dedicarse a la ciberseguridad de manera adecuada, de igual manera que en muchas otras áreas, uno necesita una buena cantidad de herramientas configuradas y entornos separados para según que actividad quiera realizar.

Para el flujo de trabajo que reconozco hasta estos momentos, distingo las siguientes necesidades:

- Lo más común que hago en ciberseguridad tiene que ver con pentesting web y en general todo el flujo de trabajo general de un pentesting: reconocimiento, análisis de vulnerabilidades, explotación y post-explotación. Para esto ocupo una colección de herramientas bastante tradicional que todos conocemos, herramientas como nmap, algún proxy como ZAP, conexión a VPNs, fuzzers, etc.
- No planeo quebrar hashes con mi máquina personal, entonces probablemente no sea necesario particularmente alguna herramienta como hashcat, pero probablemente, si encuentro una lista lo suficientemente amplia de herramientas de ciberseguridad, venga ahí por default. En ese caso, tampoco le haría el feo.

- Tal vez agradecería ciertas herramientas relacionadas a OSINT, como theHarvester, pero tampoco son mi prioridad ni mi caso de uso principal.

- Navajas suizas como metasploit y searchsploit también resultan ser bastante útiles para ciertos escenarios en los cuales no necesariamente estás analizando una aplicación o entorno en busca de vulnerabilidades, si no que las vulnerabilidades ya se encuentran descubiertas y disponibles en alguna de esas dos. Este es obviamente un caso bastante común de hecho. Es rara la vez en que descubres un CVE (pero se viene la mía próximamente :D) propio.

- Me gusta realizar ocasionalmente análisis de malware, para estos casos, obviamente primero que nada necesito una serie de entornos aislados y las herramientas pertinentes tanto para análisis estático como dinámico. Por ahora, sólo hago análisis estático para binarios hechos en Windows pero que, por si acaso, no me gustaría tener fuera de un entorno aislado de todas maneras.

Entonces, he llegado a la conclusión de que necesito al menos dos features nuevas dentro de mi flujo de trabajo para manejar todos estos requerimentos:

- Una suite de herramientas instaladas dentro de mi sistema operativo principal, destinadas a ciberseguridad.

- Una máquina virtual aislada pero también configurada de manera declarativa para correr análsis de malware aislado y para otros casos de uso en general que también requieran ese aislamiento que solamente una máquina virtual debidamente configurada puede proporcionar.

Vamos a comenzar con la primera feature:

12.1 Suite de herramientas

Usando como punto de partida el proyecto de Nix-Security Box de Fabian Affolter, voy a seleccionar un conjunto de herramientas particular para ciberseguridad, dividido en categorías. Todas las declararé como paquetes de usuario.

12.1.1 Proxies

Agregamos las siguientes herramientas de proxies para tener una buena variedad, aunque generalmente usaré solamente ZAP y BurpSuite.

```
bettercap
burpsuite
ettercap
mitmproxy
mubeng
proxify
proxychains
redsocks
rshijack
zap
firefox
```

12.1.2 Port Scanners

En general solo usaré nmap y rustscan

```
nmap
rustscan
```

12.1.3 Web

Esta sección viene extraída directamente de el proyecto de Fabian, porque necesito variedad.

```
albedo
arjun
atac
brakeman
cameradar
cansina
cariddi
cf-hero
chopchop
commix
crackql
crlfsuite
dalfox
dismap
dirstalk
nomore403
forbidden
```

```
galer
gau
genzai
gospider
gotestwaf
gowitness
graphpython
graphqlmaker
graphqlmap
graphw00f
h2spec
hakrawler
python3Packages.hakuin
hey
http-server
httpx
jaeles
jsubfinder
jwt-hack
katana
kiterunner
mantra
mitmproxy2swagger
monsoon
nikto
nomore403
ntlmrecon
offat
photon
plecost
scraper
slowlorust
snallygaster
subjs
uddup
urlfinder
urx
wad
wappalyzergo
webanalyze
```

websecprobe
whatweb
wprecon
wpscan
wsrepl
wuzz
xcrawl3r
xnlinkfinder
xsubfind3r

12.1.4 Generic

certinfo-go
chrony
clamav
curl
cyberchef
dorkscout
easyeasm
exiflooter
flashrom
girsh
gtfocli
httpie
hurl
inetutils
inxi
iproute2
iw
lynx
macchanger
nano
parted
pwgen
spyre
utillinux
wget
xh
xnldorker
btop

```
iftop
iotop
mosh
shellz
certinfo-go
cifs-utils
freerdp
net-snmp
nfs-utils
ntp
openssh
openvpn
samba
step-cli
wireguard-go
wireguard-tools
xrdp
ipcalc
netmask
tmux
zellij
cabextract
p7zip
unrar
unzip
```

12.1.5 Services

La sección de variedad por excelencia, aquí tenemos metasploit, sqlmap, etc.

```
acltoolkit
checkip
ghunt
ike-scan
keepwn
metasploit
nbutools
nerva
nuclei
nuclei-templates
```

openrisk
osv-scanner
uncover
traitor
vuls
mx-takeover
ruler
swaks
trustymail
agneyastra
ghauri
laudanum
mongoaudit
nosqli
pysqlrecon
sqlmap
sqlmc
braa
onesixtyone
snmpen
baboossh
sshchecker
ssh-audit
ssh-mitm
teler
waf-tester
wafw00f
cryptoscan
mini-pqc
pqc-bench
pqcscan
oshka
trajan
terrascan
tfsec
chain-bench
witness
davtest

12.1.6 Password

authoscope
bruteforce-luks
conpass
crunch
h8mail
hashcat
hashcat-utils
hashdeep
legba
nasty
ncrack
nth
pywhisker
sh4d0wup
spearspray
spraycharles
thc-hydra
truecrack

12.1.7 Exploits

exploitdb
go-exploitdb
keedump
padre
sploitscan
lmp
logmap

12.1.8 Fuzzers

aflplusplus
feroxbuster
ffuf
gobuster
honggfuzz
radamsa
regexploit
scout

```
ssdeep
wfuzz
zzuf
```

12.2 Maquina virtual

Para aplicativos en los que necesite una granularidad mucho más fina de lo que tenga que realizarle a mi sistema operativo. Para este aspecto de mi flujo de trabajo en ciberseguridad, busco un sistema operativo que voy a poder hacer y deshacer a mi gusto, pues esencialmente lo que busco para esta parte del flujo de trabajo es modificarlo de manera extremadamente rápida para la situación que lo amerite.

Por esto, y aprovechando las distintas capas de abstracción que existen actualmente, decidí que el mejor aplicativo para cubrir esta parte de mi flujo de trabajo es Distrobox

Distrobox es una herramienta que permite, entre otras cosas, construir máquinas virtuales de manera rápida, orientadas justamente a ejecutar herramientas en un entorno configurado, y de una manera bastante seamless. Por ejemplo, configura las VMs para que consuman el servidor gráfico y de sonido de la máquina host; lo cual permite que las aplicaciones de dentro de la VM puedan ser lanzadas dentro del entorno de escritorio de la máquina host. Pero en realidad, **Distrobox es una capa de abstracción sobre Podman**, así que habremos de instalar ambas herramientas.

Para comenzar, agrego las siguientes líneas a mi `configuration.nix`, las cuales declaran la virtualización usando Podman.

```
virtualisation.podman = {
  enable = true;
  dockerCompat = true;
};
```

Agrego mi usuario al grupo de podman, para poder correr los comandos necesarios sin problemas.

```
"podman"
```

Y finalmente, el paquete de sistema de Distrobox.

```
distrobox
```

Hecho esto, mi sistema está preparado para crear contenedores con Kali Linux y ahí crear todas las herramientas necesarias. Debido a que los contenedores me serán extremadamente volátiles y, por lo dicho anteriormente, su naturaleza será extremada imperativa y situacional a lo que necesite en este momento, los únicos comandos que guardo aquí sobre cómo interactuar con mi contenedor de ciberseguridad serán los siguientes:

12.2.1 Flujo de trabajo con el contenedor de Kali Linux

1. Creación de contenedor

   ```
   distrobox create --name kali --image docker.io/kalilinux/kali-rolling
   ```

2. Entrar al contenedor

   ```
   distrobox enter kali
   ```

12.2.2 QoL (Calidad de vida)

Para finalizar mi flujo de interacción con el contenedor de ciberseguridad que pueda crear con Distrobox, voy a agregar algunas cosillas a la calidad de vida de mi flujo de trabajo.

La primera es para crear una distinción entre la shell de mi equipo host y mis contenedores. Debido a que Distrobox y Podman montan la partición /home dentro del contenedor por default, esto significa que todas mis configuraciones de shell se trasladan a los contenedores. Y debido a que el hostname de los contenedores es idéntico al de la máquina host, esto resulta en que la shell del interior del contenedor y la del exterior resulten idénticas.

Esto me podría traer problemas muy serios en un futuro, al no poder distinguir si estoy dentro de mi contenedor o no y correr comandos que afecten a uno u otro de manera negativa por este error. Por eso, agrego la siguiente verificación a mi `.zshrc` para que cambie de color al entrar al contenedor. Si detecta la variable de entorno adecuada.

```
if [[ -n "$CONTAINER_ID" ]]; then
  export AGNOSTER_DIR_BG=magenta
fi
```

12.3 Flujo de trabajo de Ciberseguridad

Sea desde mi laptop, desde la máquina virtual, o desde la máquina del trabajo, me gusta que, también para ciberseguridad, mi ritmo de trabajo esté unificado. Para este propósito, no lo unifico bajo este proyecto pero si lo unifico bajo otro proyecto propio: PumaBlackOps.

Por esto, lo que hago en esta sección es hacer los ajustes pertinentes para este propósito. Debido a que este flujo de trabajo se cruza con un sistema operativo imperativo como lo es Kali Linux, esta es una sección de este proyecto que no puede dejar un producto final preparado, como lo hacían el resto de secciones de mi libro. En su lugar, tendré que asumir una instalación Kali Linux con mi proyecto PumaBlackOps instalado, que sourcee el `.zshrc` generado por este proyecto.

Para preparar el `.zshrc`, mi mismo proyecto PumaBlackOps implementa un plugin para OhMyZSH que hace todo el trabajo. Para esto, simplemente agregamos las siguientes lineas en el `.zshrc`

La referencia al plugin dentro de nuestra pestaña de plugins

```
pumablackops
```

Asi mismo, este proyecto se sirve fuertemente de `direnv` para poder cambiar dinámicamente de target. Por esto, agregamos también soporte para direnv en mi `.zshrc`

```
eval "$(direnv hook zsh)"
```

13 VPNs

Las VPNs son una forma de poder interactuar con equipos como si estuvieran en una red local, aunque en realidad lo estemos haciendo a través de internet. Estamos efectivamente usando la infraestructura pública ya establecida de internet para emular redes privadas. Y esto es algo muy útil para varios proyectos en equipo y para aplicaciones personales que requiero.

El servicio de VPNs que suelo utilizar más es ZeroTier One, así que lo declaro como paquete de sistema. No hay configuraciones adicionales por realizar de manera declarativa.

```
  zerotierone
```

Luego declaramos Netbird, otro servicio de VPNs que de hecho recientemente he descubierto, y muy potencialmente, sea de mis favoritos para aplicaciones en las que necesite self-hostear VPNs

```
services.netbird.enable = true;
```

Habilitamos también su interfaz gráfica, por si acaso.

```
pkgs.netbird-ui
```

Y finalizamos esta sección con tailscale, yet another servicio de VPNs que también me gusta utilizar por su facilidad de uso.

```
services.tailscale.enable = true;
```

14 Conclusión

En este documento hemos visto la configuración de mi entorno de trabajo en NixOS, el cual se basa en Hyprland como gestor de ventanas, Waybar como barra de estado, y una variedad de herramientas para mejorar mi productividad y experiencia de usuario. Además, hemos configurado herramientas para aprovechar la inteligencia artificial en nuestro flujo de trabajo, como Gptel, Aider y Copilot.el. Esta configuración me permite tener un entorno de trabajo eficiente, personalizado y adaptado a mis necesidades, manteniendo un equilibrio entre simplicidad y funcionalidad.

Finalmente, es importante mencionar que esta configuración es un punto de partida y puede ser ajustada y mejorada con el tiempo a medida que descubro nuevas herramientas y formas de optimizar mi flujo de trabajo. La flexibilidad de NixOS y la comunidad de software libre me permiten experimentar y adaptar mi entorno de trabajo según mis necesidades cambiantes.

```
}
```